Codex 蓝皮书：从项目规则到交付验证

v1.4.0 · 资料核对至 2026 年 10 月 21 日



主理人: Hunk Wu (X: @ai_pmer)

English PDF | English online

目录

第一部分：入门与环境

- Ch.01 从代码生成到结果验收
- Ch.02 安装 Codex 并核对运行环境
- Ch.03 排查沙盒与本地服务的连接

第二部分：任务与项目规则

- Ch.04 用目标和验收条件指导智能体
- Ch.05 用 AGENTS.md 记录项目规则
- Ch.06 检查执行过程并及时纠偏

第三部分：界面与移动工作流

- Ch.07 用真实界面验证前端
- Ch.08 把构建告警和审批接到手机
- Ch.09 先建立基线，再逐步改造旧系统

第四部分：重构与产品交付

- Ch.10 验证订阅制 SaaS 的支付闭环
- Ch.11 用 Expo 构建并检查移动应用
- Ch.12 从产品使用数据到获客实验

第五部分：增长与版本迁移

- Ch.13 核对 2026 年的模型与工具变更

Ch.01 从代码生成到结果验收

> ***问题***：*把 AI 当作自动补全，或直接接受未检查的改动，容易漏掉需求和回归问题。*
>
> ***本章实践***：*用目标、边界和验收条件写一个可执行的开发任务。*

在“实战产品说”与很多读者交流时，我发现大家最容易陷入一个思维陷阱：拼命去背各种 AI 编程工具的快捷键和底层语法，像背 API 手册一样去学习 AI 编程。

Codex 等编码智能体可以承担样板代码、测试和资料整理，但需求判断、关键设计和结果验收仍需开发者负责。相比记住更多提示词，先学会描述目标、限定修改范围并检查结果更有用。

本章用一个任务模板说明如何把需求转成可执行、可验证的开发工作。

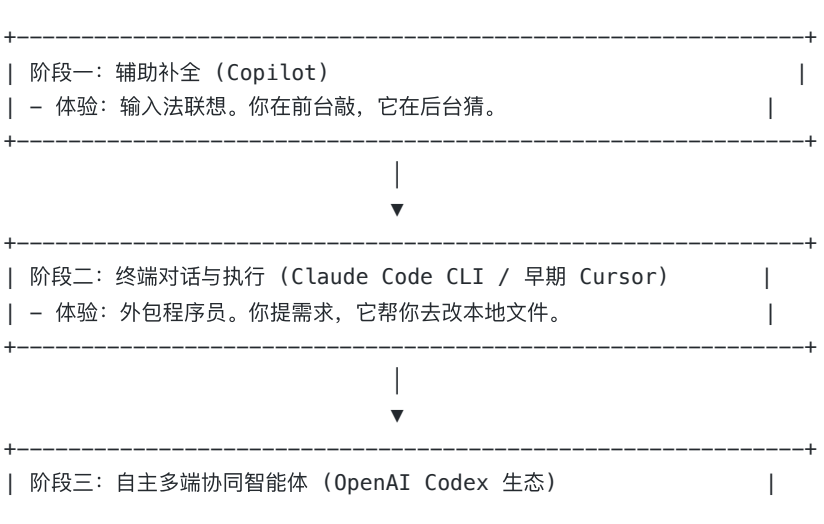
理解方式：告别砖瓦工，做工地总建筑师

如果你第一次接触 AI 编程，不要把它想成一个冷冰冰的代码生成器。用生活中的场景来理解：

```
【传统手写代码】 ──> 你自己当泥瓦匠，一砖一瓦搬砖砌墙，稍不留神墙角砌歪了，整栋楼推倒重来。
【传统 AI 补全】 ──> 你在前面砌砖，AI 在后面帮你递砖头（你敲一行，它猜半行，手脚依然停不下来）。
【Codex 编排时代】──> 你是「总承包建筑师」，Codex 是「高素质施工队」：
                        - 你画蓝图（定义目标 Goal）
                        - 你立施工安全围挡（设定沙盒与红线 Constraints）
                        - 你用水平仪验收房子（自动化测试 Validation）
                        - 施工队（Codex）自己去算力学结构、买水泥、砌墙、抹灰。
```

当心态从“我怎么敲出这段逻辑”变成“我怎么向施工队下发一份不会跑偏的施工单”时，你才算真正跨入了 AI 原生研发的大门。

1.1 编程浪潮的演进：我们在哪里？



```
| – 体验：数字化全栈研发团队。目标驱动、沙盒运行、跨端协同。          |
+-------------------------------------------------------------+
```

很多人仍把时间消耗在第一或第二阶段，但 2026 年的 Codex 生态已经确立了第三阶段标准：

- **跨端生产力矩阵**：本地 CLI 自动化执行，ChatGPT 桌面端（代码模式）负责复杂审查与 Sites 原位预览，手机端全天候离线看护与 Guardian 审批，辅以 Desktop Computer Use 自主操作浏览器完成视觉走查。
- **强推理与前沿智能体内核**：从复杂跨工具任务所用的 **GPT-6 Astra**，到日常复杂编码的 **GPT-6.1 Sol** 与明确小任务所用的 **GPT-6 Luna**，按任务选择模型，再在自己的环境中核对工具、权限与实际结果。[当前模型说明]

1.2 核心心智转变：从“过程控制”到“边界控制”

做产品经理（PM）最深的心得往往是：给研发下需求，最忌讳说“第一步点这个按钮，第二步去数据库查那张表”——那是典型外行指导内行。优秀的做法是给出标准 **PRD（产品需求文档）**：讲清业务目标、输入输出规格、性能边界与验收测试用例。

指挥 Codex 完全同理。过去你用 Prompt 教 AI 怎么写 for 循环（过程驱动）；现在你要切换为架构师角色，只抓好三件事：

1. **定义终态 (Goal)**：这个功能要替终端用户解决什么具体问题？
2. **确立红线 (Constraints)**：哪些文件不可修改？安全密钥如何隔离？沙盒等级如何圈定？
3. **自动化验证 (Validation)**：提供可运行的测试命令或断言，让 Codex 自主运行并证明其产出的可靠性。

> *[!IMPORTANT] **主理人硬核公式***
> *`稳定交付 = 明确的终态 (Goal) + 严苛的红线 (Constraints) + 自动化的断言 (Validation)`*

动手实践：先完成这 3 步

刚进入项目的新手，切忌一上来就给 AI 丢一段长达千字的模糊想法。按以下 3 步执行最简单的闭环：

1. **步骤一：起草 50 字极简需求卡**
   明确三要素：做哪个页面（目标）、基于现有哪个框架（约束）、用什么命令验证成功（断言）。
2. **步骤二：给 Codex 划定安全活动范围**
   在终端中使用沙盒模式执行（如 `codex exec --sandbox workspace-write`），禁止它越权访问外部系统。
3. **步骤三：让 AI 运行测试自证清白**
   要求 Codex：“修改完成后自行运行 `npm run build`，直到终端退出码为 0 再交付”。

🚨 翻车自救与避坑速查表

常见踩坑现象	致命原因	翻车自救锦囊
把 AI 当百度搜索问概念	对话几小时全是文字理论，没有落地一行代码	立即终止闲聊，直接给出一个目标：“在 `src/` 下创建登录组件并跑通测试”
一次性喂入上万行无关代码	上下文被噪音撑爆，AI 发生注意力涣散并胡编滥造	明确指出关心的 1-2 个具体文件路径，让 AI 局部精准重构
不看测试只信 AI 嘴上说“已完成”	AI 以为自己改对了，实际项目根本跑不起来	在 Prompt 强制追加验收断言：“修改后必须在终端执行构建验证，输出通过日志”

1.3 你的新护城河：编排力与商业认知

如果手写语法不再是技术壁垒，那么独立开发者与技术产品经理的真正护城河是什么？

在 pmer.cn 上我反复强调：**壁垒在于你对业务边界的定义，以及你对用户同理心的理解。**

你不再需要花三天去调试一个 CSS 弹性盒居中，或者翻遍 StackOverflow 解决打包构建报错；你的核心任务，是运用 Codex 这套多端协同矩阵，快速组装出可落地的商业 MVP，把产品推向真实市场去验证用户付费意愿。

在接下来的章节中，我们将正式开启这套“跨端编排”的落地之旅。

Ch.02 安装 Codex 并核对运行环境

> ***问题***：*CLI、依赖或权限配置不一致时，任务会在运行前失败。*
>
> ***本章实践***：*安装 CLI，核对版本与登录状态，再检查桌面和移动入口的可用条件。*

工欲善其事，必先利其器。在“实战产品说”中，我常强调一个原则：**AI Native 开发的第一步，是把你的“指挥舱”配置得足够稳定。**很多新手急着去跟 AI 聊天写代码，结果因为本地环境不匹配、权限没开对，导致 AI 在终端疯狂报错、浪费 Token。

本章将带你一步步配置 Codex 的多端生产力矩阵，包括 Codex CLI 客户端、ChatGPT 桌面端代码模式以及 ChatGPT 移动端的联动桥接。

理解方式：打造你的“三端作战指挥舱”

不要把各个端当成孤立的软件。想象你正在指挥一支太空探险队：

```
【Codex CLI】              —> 主机舱的「动力引擎」：挂载在终端底层，负责高并发、纯代码批处理、跑测试与 CI 构建。
【ChatGPT 桌面端代码模式】—> 舰桥的「全景战术大屏」：多仓库 Diff 审查、Sites 页面原位预览、Computer Use 视觉走查。
【手机 ChatGPT App】      —> 舰长的「随身呼机」：离开工位时监控构建状态，收到需要人工处理的操作提醒（须另行接入审批系统）。
```

三端各司其职，你就不必死守在工位前按键盘，实现全天候随时随地编排交付。

动手实践：先完成这 3 步

如果你是第一天上手，不需要做复杂的网络穿透或高级脚本配置，按以下 3 步即可 5 分钟跑通首个任务：

1. **步骤一：安装 CLI 并完成账号鉴权**
   先确认已安装 Node.js 和 npm，再在终端运行：

   ```
   npm install -g @openai/codex@latest
   codex
   ```

   浏览器将自动弹出 ChatGPT 登录窗口，登录你的账号完成快速绑定。
2. **步骤二：确认版本并选择当前模型** 先运行 `codex --version`。截至 2026 年 10 月 21 日，官方更新日志列出的 CLI 版本是 `0.163.0`；本机版本可能不同。更新日志 在账号可用时，用以下命令启动 GPT-6.1 Sol；现有 `~/.codex/config.toml` 不会被覆盖：

   ```
   codex --version
   codex --model gpt-6.1-sol
   ```

   若希望设为本地默认模型，在现有 `~/.codex/config.toml` 中设置：

   ```
   model = "gpt-6.1-sol"
   ```
3. **步骤三：用安全沙盒模式跑通第一个任务**
   进入你的项目根目录，运行：

```
codex exec --sandbox workspace-write "检查当前目录代码规范并输出修复建议"
```

2.1 CLI 客户端安装与版本核对

Codex CLI 核心由高效的 Rust 编写（codex-rs），但 OpenAI 通过 npm 提供了官方封装分发，**作为用户你完全不需要安装 Rust 工具链**，通过 npm 安装时，先用 `npm view @openai/codex@latest engines` 核对当前包的 Node.js 要求。

1. 基础环境检查与安装

在终端运行：

```
# 1. 检查 Node.js 和当前 npm 包的版本要求
node --version
npm view @openai/codex@latest engines

# 2. 安装当前最新稳定版 CLI；版本以官方更新日志为准
npm install -g @openai/codex@latest

# 或使用官方一键脚本（macOS / Linux）
curl -fsSL https://chatgpt.com/codex/install.sh | sh
```

Windows 用户可通过 PowerShell 安装：

```
powershell -ExecutionPolicy ByPass -c "irm https://chatgpt.com/codex/install.ps1 | iex"
```

安装完成后验证版本：

```
codex --version
```

2. 认证方式选择：ChatGPT 账号 vs API Key

Codex CLI 提供两种认证方式，**官方默认推荐 ChatGPT 账号登录**：

- **方式 A（推荐 / 默认）**：直接在终端运行 `codex`，浏览器会自动弹出 ChatGPT 登录页面。**ChatGPT Plus（$20/月）、Pro、Team、Business、Edu、Enterprise 套餐都已经包含 Codex 用量额度**，对独立开发者是性价比最高的选择。
- **方式 B（按量计费 / CI 场景）**：使用 OpenAI API Key。适合 CI/CD 自动化流水线或没有浏览器交互的服务器。

使用 API Key 登录时，按 Codex CLI 官方说明从安全存储中读取密钥，不要把真实密钥写入仓库或教学示例。

3. 账单熔断防爆配置

很多新手最担心的就是 AI 暴走刷爆信用卡。Codex 提供多道用量与权限控制：

1. **OpenAI 后台硬上限（最关键）**：在 OpenAI 开发者后台为该 API Key 设置月度 Usage Limit 硬限制（新手建议设为 $50）。即使 AI 陷入异常死循环，也能确保你的资金万无一失。
2. **模型与任务匹配**：在 Codex 中选择实际可用的模型，日常复杂编码可从 `gpt-6.1-sol` 开始；API 账单与 Codex 订阅用量分别核对。
3. **沙盒与自动审批**：新的非交互调用使用 `--sandbox workspace-write`；`--approve-for-me` 是独立的自动审批模式，不能保证所有请求获准，也没有官方固定的 Luna 后台模型。权限说明

4. 可选：评估第三方供应商切换工具

如果要在不同模型供应商之间切换，先确认目标供应商的 API 兼容性和数据处理方式。**Codex Switch** 是本书作者维护的独立开源工具，不属于 OpenAI 官方 CLI。其项目 README 记录了适用版本、配置步骤和对本机会话文件的修改方式；跨供应商续聊不能对未来版本作保证。

按该仓库当前 README，入口脚本是 `codex-switch.sh`，并不存在 `install.sh` 或 `codex-switch --to` 命令。先阅读 README、备份本机会话与配置，再按需要配置供应商并运行脚本。以下只核对下载后的入口文件，不执行切换：

```
git clone https://github.com/aipmer/codex-switch.git
cd codex-switch
test -f codex-switch.sh && sed -n '1,35p' README.md
```

2.2 桌面端：ChatGPT 桌面客户端「Codex 代码模式」

> ⚠️ ***重要生态演进**：2026 年下半年，原独立的 Codex 桌面应用正式退役，所有桌面端能力已完全并入 **ChatGPT 桌面客户端**。*

1. 安装与进入代码模式

```
# macOS：直接下载 ChatGPT.dmg，拖入 Applications
# Windows：使用 winget 一键安装
winget install OpenAI.ChatGPT
```

启动 ChatGPT 客户端并登录后，点击左侧边栏的 **Codex** 标签页，即进入「Codex 代码模式（Code Mode）」。

新增核心生产力特性：

- **Sites 原位托管**：在客户端内一键创建、部署和预览 Web 项目，配合 Annotations 批注实现“指着界面改代码”。
- **多仓库审查（Multi-repo Review）**：大型微服务或多包工程可在单一视图下统一审查 Diff。
- **内置浏览器升级**：直接检索浏览历史，并可引用当前 Chrome 打开的标签页进行上下文分析。

2. Computer Use 视觉测试权限配置

让 AI 自主操作桌面屏幕（Computer Use）**目前主要支持 macOS**，需在 macOS「系统设置 → 隐私与安全性」中授予 ChatGPT 两项核心权限：

```
[macOS 系统设置] -> [隐私与安全性]
  ├─ 辅助功能 (Accessibility) ──> 勾选 [ChatGPT] (允许模拟鼠标点击/按键)
  └─ 屏幕录制 (Screen Recording) ──> 勾选 [ChatGPT] (允许截屏并由 Vision 模型分析)
```

安全边界控制：

- 首次操作某个第三方应用（如 Google Chrome）时，会弹窗询问你是否授权该应用。
- ChatGPT 无法自动操作终端本身、自身界面或系统级管理员授权弹窗（例如 `sudo` 提权输入密码）。

2.3 手机端（ChatGPT App）全天候看护桥接

人在户外或离岗时，无需死守工位：

1. **官方原生任务同步**：当你使用 ChatGPT 账号登录 Codex 时，终端与云端的任务都会实时同步到手机 ChatGPT App 中。你可以掏出手机查看长任务的执行进度、随时发送补充指令。
2. **Guardian 审批与 Webhook 提醒**：结合 Ch.08 的移动看护网关，当遇到需要人工确认的临界高危操作（如线上部署、数据库变更）时，可通过飞书等渠道发送告警；真正的放行或驳回需要经过身份验证、任务绑定和执行结果检查。

翻车自救与避坑速查表

常见报错 / 翻车现象	核心原因	排查与处理
`--full-auto` 显示弃用警告	旧调用仍走兼容入口	新的非交互调用使用 `--sandbox workspace-write`

`SyntaxError: Unexpected token ...` (Node 报错)	本地 Node.js 版本低于 20	运行 `node -v` 检查，使用 `nvm use 20` 或 `nvm install 20` 升级 Node
`Permission denied: Screen Recording`	macOS 未开启屏幕录制权限	打开「系统设置 → 隐私与安全性 → 屏幕录制」，将 ChatGPT / Terminal 勾选开启并重启客户端
`gpt-5.4` 在 Codex 登录态不可用	该模型于 2026-08-31 从 Codex 的 ChatGPT 登录态退役	检查 `~/.codex/config.toml`，在账号可用时选择 `gpt-6.1-sol`

当前模型与迁移详解 · Codex 工具频道上手指南

Ch.03 排查沙盒与本地服务的连接

> ***问题***：*隔离环境中的 localhost 不一定指向本机数据库，连接失败需要先确定运行位置。*
>
> ***本章实践***：*用连接测试定位网络边界，按需要配置受控隧道。*

使用 Codex 运行数据库测试时，新手最常碰到的灵异报错是：明明本地 Docker 里的 PostgreSQL 跑得好好的，AI 却在终端狂喊 `Connection refused to localhost:5432`。

先确认 Codex 命令运行在本机、容器还是远端环境：不同环境的 `localhost` 指向不同机器。本章用连接测试定位问题，并在确有需要时配置受控访问。

理解方式：防爆隔离仓与专用生命导管

把 Codex 的执行环境想象成医院里的「高等级无菌防爆隔离仓」：

【你的宿主机 Mac/PC】 —> 外部大世界：装着你真实的本地数据库、私钥配置、个人微信和文件系统。
【Codex 运行沙盒】 —> 无菌隔离仓：AI 所有的代码编译、文件生成都在这个密闭盒子里运行，文件访问范围取决于实际沙盒配置和审批模式，修改前应核对权限。
【端口穿透与反向隧道】—> 密封手套与气体导管：当隔离仓里的 AI 需要连外界的本地数据库时，你必须主动在舱壁接一根“专用导管”（把端口转接到仓内）。

如果不接这根导管，AI 在隔离仓里自言自语喊 `localhost`，找到的只是空无一人的仓壁，自然就会报错 `Connection refused`。

动手实践：先完成这 3 步

如果你需要让沙盒连上本地数据库，按以下 3 步建立通道：

1. **步骤一：确认本地服务端口正处于监听状态**
   在终端确认 Docker 或本地服务正常运行（如 Postgres 在 5432 端口）：

   ```
   lsof -i :5432  # 查看端口是否有进程正在监听
   ```

2. **步骤二：启动极简端口穿透（使用 ngrok 或 SSH）**
   运行一条命令为 5432 端口打通公网安全临时通道：

   ```
   ngrok tcp 5432
   # 获得转发地址，形如：tcp://0.tcp.ngrok.io:12345
   ```

3. **步骤三：将穿透连接串注入 Codex 沙盒执行任务**
   在执行时将地址作为环境变量传入，让 AI 执行连接测试：

```
export DATABASE_URL="postgresql://postgres:password@0.tcp.ngrok.io:12345/dev_db"
codex exec --sandbox workspace-write "运行数据库连通性自测脚本 npm run test:db"
```

3.1 沙盒网络壁垒：为什么 `localhost` 连不上？

默认状态下，沙盒与本地宿主机处于安全网络阻断状态：

```
+-----------------------------------+                  +-----------------------------------+
|     云端沙盒 (Sandbox)      |                  |    宿主机 (Local Mac)      |
| - App Code                  |    (隔离屏障)    | - Docker container         |
| - localhost:5432 (空无一人) ---[ X ]-----------> | - PostgreSQL (port: 5432) |
+-----------------------------------+                  +-----------------------------------+
```

在当前 Codex CLI 中，沙盒边界由权限配置决定：

- `--sandbox read-only`：只读模式，智能体只分析代码，不具备写权限。
- `--sandbox workspace-write`（推荐常用）：智能体可以修改当前项目目录，但不可随意跨出目录或篡改全局系统。

要让沙盒内的代码读写宿主机上的数据库或微服务，必须进行**端口映射与反向隧道穿透 (Reverse Tunneling)**。

3.2 实战：使用 SSH / Ngrok 建立反向隧道

方法一：使用 SSH 反向端口转发（推荐长效方案）

如果你有公网 VPS 服务器，SSH 反向转发是最安全且免费的方案：

```
# 将本地的 5432 (Postgres) 转发到公网中转机的 54320 端口
ssh -R 54320:localhost:5432 user@your-public-vps.com -N
```

接着在环境中配置：

```
export DATABASE_URL="postgresql://postgres:password@your-public-vps.com:54320/dev_db"
```

方法二：使用 Ngrok 极速穿透（零服务器方案）

宿主机终端直接运行：

```
ngrok tcp 5432
```

终端输出类似：`Forwarding tcp://0.tcp.ngrok.io:12345 -> localhost:5432`。

将该临时地址写入环境变量或注入 Codex 会话即可。

3.3 目录映射与环境变量同步规范

1. 密钥安全硬隔离

严禁让 AI 自动把包含真实生产密钥的 `.env` 提交到仓库或云端。在 `.gitignore` 与项目 AGENTS.md 中添加硬约束：

```
## 🔴 Hard Constraints
- Never sync, copy, or commit files matching *.env.
- Always use `src/config.ts` or environment variables for secret injection.
```

2. 沙盒临时缓存清理

为防止 node_modules 缓存或 build 缓存导致的幽灵构建故障，在 AGENTS.md 中写入标准化清理脚本：

```
## 💻 Developer Commands
- **Clean Run**: `rm -rf node_modules/.cache .next/cache && npm run test`
```

🚦 翻车自救与避坑速查表

常见报错 / 翻车现象	致命原因	排查与处理
`Connection refused to 127.0.0.1:5432`	沙盒以为本地有数据库，实际隔离舱内空无一物	检查反向穿透隧道是否开启，严禁用 `localhost`，必须使用穿透地址
`ngrok session expired / reconnecting`	免费版 ngrok 隧道中断或地址重置	重启 ngrok 复制新地址，或改用项目自带的 `scripts/codex-watchdog` 反向穿透工具
`EACCES: permission denied`	沙盒权限受限，试图写入项目外的受保护系统目录	检查命令是否使用了正确的 `--sandbox workspace-write`，并约束操作在项目根目录内

Ch.04 用目标和验收条件指导智能体

> ***问题***：*模糊需求容易让智能体自行补全错误假设；过细步骤又限制实现空间。*
>
> ***本章实践***：*编写包含目标、范围、限制和可运行验收命令的任务说明。*

无论使用哪款模型，只有步骤却没有验收条件的任务都难以判断是否完成。对有多个可行实现的需求，先写清目标、限制和验收方式，再让智能体选择实现细节。

本章教你如何用专业产品 Specs 的方式，精准指挥 Codex。

理解方式：米其林主厨与点餐下单单

很多人给 AI 派任务，就像在米其林餐厅吃饭时，冲进后厨对大厨指手画脚：

【保姆式命令（过程驱动）】 —> ❌ “大厨，你左手拿刀，把土豆切成 2mm 细丝，锅烧到 180 度倒 15ml 油，翻炒 3 分钟，最后放 3g 盐。”

（大厨觉得你在侮辱他的厨艺，而且一旦煤气火候微变，菜必糊无疑。）

【架构师下单（目标驱动）】 —> ✅ “大厨，我要一份香煎土豆做牛排配菜。

- 目标（Goal）：口感香脆，作为晚宴配菜；
- 红线（Constraints）：热量低于 200 卡，严禁使用黄油与花生油（客人过敏）；
- 验收（Validation）：15 分钟内出菜，中心温度达到 75℃。”

Codex 就是精通算法和架构的“米其林主厨”。你要做的是告诉它吃什么、什么不能碰、怎么算合格，具体的刀工和火候完全交给它自己规划。

动手实践：先完成这 3 步

起草你的第一份目标驱动 Specs：

1. **步骤一：定义终态 (Goal)**
   用一句话说清最终交付物，如："在 `/api/checkout` 实现 Stripe 支付会话创建接口"。
2. **步骤二：划定绝对红线 (Constraints)**
   写明 2-3 条不能触碰的规则，如："严禁明文打印 Secret Key，禁止修改全局中间件"。
3. **步骤三：提供自动校验断言 (Validation)**
   给出一个可运行的命令，如："运行 `npm test -- checkout.test.ts` 必须一次性全绿通过"。

4.1 目标驱动的 Markdown 结构规范

在终端或 ChatGPT 桌面端中向 Codex 发起任务时，建议使用标准 Markdown 结构：

```
# 🎯 Goal
[描述希望达到的最终业务状态。例如：实现一个支持 GitHub OAuth 登录并能持久化存储偏好的路由。]

# 🛑 Constraints (红线约束)
- [安全约束：绝对不能将密钥明文写入代码或日志中。]
- [选型约束：必须沿用现有 Tailwind，禁止额外引入新的 UI 库。]
- [防腐约束：严禁改动 src/legacy 目录下的任何历史文件。]

# 🧪 Validation Specs (验收断言)
- [自动化测试：运行 npm run test:unit 测试套件必须 100% 通过。]
- [边界异常：当输入字段缺失时，接口必须返回 400 Bad Request 并附带标准 JSON 错误说明。]
```

4.2 真实案例对比：传统 Prompt vs 目标驱动 Specs

假设我们要写一个 **"带 Redis 限流的 API 代理服务"**。

❌ 传统过程驱动型 Prompt

> *"请帮我用 Express 写一个 API 代理。首先引入 express 和 express-rate-limit。然后配置 rate-limit，设置 windowMs 为 15 分钟，max 为 100 次。接着写一个路由 `/api/proxy`，使用 axios 请求第三方 API `https://api.github.com`。如果请求成功就返回数据，如果失败就返回 500 错误。注意在请求头里带上 Authorization Bearer Token。"*

✅ 目标驱动 Specs

```
# 🎯 Goal
实现一个 Express API 代理路由，安全代理所有发往 GitHub API 的请求。

# 🛑 Constraints
- 必须使用 Redis 作为限流器的数据源，禁止使用内存型限流，以支持多容器横向扩展。
- 代理请求的超时时间（Timeout）必须硬性限制在 3000ms 以内，防止挂起 Node 事件循环。
- 严禁将 GitHub Token 写入代码或日志，必须从 `process.env.GH_TOKEN` 安全读取。

# 🧪 Validation Specs
- 压测模拟：1 分钟内发送超过 60 次请求时，代理必须返回 429 Too Many Requests。
- 容错断言：当上游发生超时或网络断开时，必须返回 504 Gateway Timeout 并包含结构化 JSON。
```

4.3 GPT-5.6 Terra 对 Specs 的推理链路分析

当你把这套规范交给 Codex 后，其内部推理过程会这样高效拆解：

```
[解析 Specs 目标] ──> 分析 Constraints (Redis限流 / 3s超时 / 密钥隔离)
                              │
                              ▼
[自主选型与架构规划] ──> 引入 ioredis + rate-limit-redis，配置 Axios 拦截器与超时断路器
                              │
                              ▼
[对齐 Validation Specs 编写断言自测] ──> 模拟 429 频控场景 + 模拟超时场景并验证
                              │
                              ▼
[自愈代码直至测试 100% 通过] ──> 提交最终干净的代码与测试报告
```

把逻辑规划权让渡给 AI，把验收标准牢牢攥在自己手里。 这就是 AI 时代最高效的人机协同法则。

🚦 翻车自救与避坑速查表

常见踩坑现象	致命原因	排查与处理
步骤写太碎，AI 遇到异常直接卡死	过程命令剥夺了 GPT-5.6 的自愈规划空间	删掉具体实现步骤，只保留目标、红线与测试断言，放手让 AI 规划
AI 改完后产生连锁反应改坏其他模块	没有设定明确的代码防腐与修改范围 Constraints	在 Specs 中严厉声明红线：“仅允许修改 `src/modules/auth/`，严禁碰触其他目录”
AI 以为写完了，其实逻辑存在致命破绽	缺少自动执行的断言（Validation Specs）	要求它在终端自行运行 `npm run test` 或编写单元测试覆盖边界条件

Ch.05 用 AGENTS.md 记录项目规则

> ***问题***：*智能体缺少项目约定时，可能修改无关文件或重复尝试失败命令。*
>
> ***本章实践***：*建立可维护的 AGENTS.md，并区分文字规则与真实权限、沙盒约束。*

在实际做产品的过程中，最怕遇到的一种开发情况是：修好了一个 Bug，却顺手带出了三个新 Bug；或者新写了一个功能，结果把团队约定的代码风格破坏得一塌糊涂。

在项目根目录放一份 ** `AGENTS.md` **，可以让 Codex 读取技术栈、常用命令和协作约定。重要操作仍要通过权限配置、代码审查和测试控制。

本书把这套写法称为 **Codex Collaboration Protocol (CAP)**；它是项目约定，不是 Codex 的内置安全机制。

理解方式：新员工入职交接手册与安全守则

把 Codex 想象成刚刚入职你公司的“天才实习生”：

```
【没给 AGENTS.md】 ──> 天才实习生第一天上班，没人告诉他规矩，他为了修一个前台样式，
                       顺手把你运行了三年的核心认证模块全重构成他喜欢的非标写法，上线直接炸库。
【配好 AGENTS.md】 ──> 天才实习生一进门，桌上放着打印好的《团队入职守则》：
                       - “这是 Next.js 15 项目，用 Tailwind”（明确技术栈）
                       - “编译运行 npm run build，别瞎改命令”（规范标准动作）
                       - “绝对禁止动 auth/ 鉴权目录，违者开除”（划定绝对红线）
                       实习生知道先检查什么，但交付前仍需审核和测试。
```

`AGENTS.md` 可以减少重复交代项目背景，但内容必须与当前代码和命令保持一致。

动手实践：先完成这 3 步

从下面三步建立第一份项目规则：

1. **步骤一：在项目根目录创建文件**
   在终端运行：

   ```
   touch AGENTS.md
   ```

2. **步骤二：填入 4 大核心板块极简模板**
   将以下内容存入 `AGENTS.md`：

   ```
   # 🤖 Codex Collaboration Protocol

   ## 📌 Project Fingerprint
   - Stack: Next.js 15, TypeScript, Tailwind CSS

   ## 💻 Developer Commands
   - Build: `npm run build`
   - Test: `npm run test`

   ## 🔴 Hard Constraints
   - Never update package.json dependencies without human confirmation.
   - Never touch files inside `src/legacy/`.
   - Always run `npm run build` before completing the task.
   ```

3. **步骤三：启动 Codex 验证加载**
   运行 `codex` 后，要求它复述本项目的构建命令和禁止修改的路径，核对回答与文件内容一致。

5.1 为什么我们需要 `AGENTS.md`？

`AGENTS.md` 是 OpenAI Codex 官方约定的指令文件名。Codex 在启动时会从当前工作目录开始，**逐级向上扫描**并合并以下文件，作为工作区指令的一部分：

1. `~/.codex/AGENTS.override.md`（最高优先级，个人覆盖）
2. `~/.codex/AGENTS.md`（全局指令）
3. 项目根目录的 `AGENTS.md`（团队约定）
4. 当前工作目录的 `AGENTS.md`（最具体的局部约定）

它的核心价值在于：

1. **共享项目约定**：在适用目录中提供技术栈、命令和协作边界，减少重复说明。
2. **标明修改边界**：说明哪些路径需要先征询或额外审查；仅靠文字不能强制阻止文件修改。
3. **指明验证命令**：给出当前可运行的构建和测试命令；敏感命令还需独立权限控制。

5.2 `AGENTS.md` 的核心四大版块

```
# 项目指纹（Project Fingerprint）
- 告诉 AI 这是一个什么样的项目，核心技术栈是什么。

# 开发常用命令（Commands）
- 明确指出编译、测试、迁移数据库的命令，不要让 AI 瞎猜。

# 架构与编码规范（Styles & Patterns）
```

```
- 规定文件存放目录、大小限制及必用的设计模式。

# 智能体安全红线 (Hard Rules)
- 绝对的禁区。一旦触碰，Codex 必须立刻中止并请求人工确认。
```

5.3 实战：一份可按项目修改的 `AGENTS.md` 示例

以下是一个典型的全栈 SaaS 项目的 `AGENTS.md` 规范：

```
# 🤖 Project: Aurora SaaS Core (AGENTS.md)

## 🧬 Project Fingerprint
- **Stack**: Next.js 15 (App Router), TypeScript, Prisma ORM, TailwindCSS.
- **Database**: PostgreSQL on Supabase.
- **Auth**: Next-Auth v5.

## 💻 Developer Commands
- **Install**: `npm install` (Only run if package.json has changed)
- **Dev Server**: `npm run dev`
- **Lint Code**: `npm run lint`
- **Run Tests**: `npm run test`
- **DB Migration**: `npx prisma migrate dev` (Never run in production branch)

## 🎨 Styles & Architecture Patterns
- **Directory Structure**:
  - Components: Keep UI components inside `@/components/ui/` (Shadcn styled).
  - Business Logic: Custom React hooks must go to `@/hooks/`.
  - API Routes: Next.js Route Handlers go to `src/app/api/.../route.ts`.
- **Formatting**:
  - Keep components modular. If a component exceeds 200 lines, decompose it.
  - All API routes must implement Zod schema validation for request body.
  - Return HTTP 400 for validation errors, 500 for internal uncaught errors.

## 🛑 Agent Boundary & Hard Rules
- **READ-ONLY Directories**:
  - Never modify files inside `src/app/api/auth/[...nextauth]` (OAuth Core).
  - Never alter `prisma/schema.prisma` without explicit human confirmation.
- **PR Rules**:
  - Before declaring a feature complete, run `npm run test` and `npm run lint`.
  - If tests fail, rollback the change immediately and report the error logs.
- **Security Check**:
  - Never commit raw `.env` files or API Keys. Use environment variables.
```

5.4 规则、沙盒与 Hooks 各自负责什么

`AGENTS.md` 提供项目上下文与行为要求，不能替代文件系统权限。需要限制可写范围时，使用 Codex 沙盒；需要审查越界请求时，选择人工审批或 `--approve-for-me` 自动审查。Hooks 可在受支持的事件上运行检查，但要先检查来源与信任状态。[权限说明](#) · [Hooks 文档](#)

```
codex --sandbox workspace-write "Review the current changes and report issues"
```

以下配置示例选用 GPT-6 Sol，并在任务结束时运行当前仓库的空白错误检查。`Stop` 事件与处理器都使用数组结构；启用后在 CLI 中用 `/hooks` 核对信任状态：

```toml
# ~/.codex/config.toml
model = "gpt-6-sol"

[[hooks.Stop]]

[[hooks.Stop.hooks]]
type = "command"
command = "git diff --check"
```

模型选择、沙盒权限和 Hook 检查是三项独立设置。Hook 的检查结果不能替代构建与测试。

🚦 翻车自救与避坑速查表

常见踩坑现象	致命原因	排查与处理
规则写得密密麻麻，AI 还是偶尔犯规	单个文件字数超标或存在相互矛盾的条目	精简规则，优先写“负向禁止（Never do X）”；关键安全规则升级为沙盒/Hooks 硬拦截
AI 陷入同一 Bug 连续自我修改 5 次死循环	缺少 Anti-Loop 防死锁机制	在 AGENTS.md 写入：“连续尝试修复同一错误超过 2 次必须立即停下，输出根因并等待指令”
AI 擅自安装各种乱七八糟的 npm 依赖	允许其随意执行包管理器命令	在红线中严令：“禁止在未获批准的情况下运行 npm install 添加新 package”

Ch.06 检查执行过程并及时纠偏

> ***问题***：*任务持续运行时，错误假设可能扩大改动范围。*
>
> ***本章实践***：*查看可见的计划、命令和差异；发现偏离后中断并给出具体限制。*

在传统开发中，管理初级程序员时最让人头疼的情景，莫过于他闷头闭门造车一周，最后交付了一堆与业务方向南辕北辙的代码，甚至把主干分支改崩。

无论使用哪款模型，任务范围和前置假设都可能出错。长任务应定期检查可见计划、工具调用和实际差异。

本章说明如何根据可见的执行记录发现偏离，并用明确的限制纠正任务。

理解方式：在 AI 脑海里装一个“监考透视窗”

不要把 AI 推理当成一个黑盒魔术：

```
【被动盲等模式】 —> 就像期末考试，你坐在考场外干等 2 小时，交卷后才发现学生从第一道题就把公式背错了，整张试卷全判零分。
【透视纠偏模式】 —> 就像你在考场里站在学生身后，看着他面前的「草稿纸」(Reasoning Summary)：
                    - 他刚在草稿纸写下：“假设要重写整个数据库 Schema……”
                    - 你立刻轻轻拍拍他肩膀：“别动 Schema，只改当前的查询索引！”
                    - 学生瞬间在草稿纸划掉错误思路，重新回到正轨。
```

推理摘要可帮助理解当前方向，但仍应以实际命令、文件差异和测试结果为准。

动手实践：先完成这 3 步

用 3 步学会像技术总监一样监督并纠正 AI：

1. **步骤一：启动交互式 TUI 并锁定 Reasoning 面板**
   在终端运行 `codex`，观察当前版本提供的状态摘要、工具调用和输出。
2. **步骤二：发现假定走偏立即按下 `Ctrl + C`**
   一旦在思考流中看到它打算引入陌生外部依赖或重构核心非目标文件，立即按下 `Ctrl + C` 中断。
3. **步骤三：一句话精准纠偏并恢复会话**
   直接输入修正提示词："禁止重构已有数据表，改用内存缓存解决"，然后要求它先复述新的限制，再继续修改。

6.1 为什么要看模型的推理过程?

Codex 界面可能显示模型生成的推理摘要和执行状态；具体呈现随客户端与模型而变。可以用这些信息检查方向，但它们并不等于完整的内部推理。

```
[用户需求] ──> 1. 解析目标与限制 ──> 2. 规划步骤 ──> 3. 运行测试 ──> 4. 自我修正 ──> [最终输出]
                  └──(在 TUI 中显示为 Reasoning Summary，即你的"监考视窗")──┘
```

> ⚠️ ***注意**：完整的思维链（Chain of Thought）为安全防护内部流，TUI 中看到的是模型自主生成的**推理摘要**。它只能作为线索，技术选型还应通过代码差异与验证结果确认。*

6.2 如何在 TUI 中观察并解读推理过程

1. 交互模式（TUI）

运行 `codex` 进入 TUI。命令会随版本变化，先用 `/help` 核对可用指令，再查看差异或发起审查：

```
# TUI 交互快捷命令
/diff       # 查看当前会话生成的真实代码变动
/review     # 唤起子智能体自动审查最近的代码变更
/copy       # 快速复制最后一条响应代码
```

2. 非交互模式（脚本化分析）

在后台运行长任务时，可通过 JSONL 流进行监控：

```
# 非交互模式，输出结构化 JSONL 流
codex exec --json "重构认证中间件" > task.jsonl
```

实时推理摘要示范

当 Codex 收到"修复 Redis 限流器连接超时"时，Reasoning 面板的健康思考流通常如下：

```
[Reasoning Summary - 正常流]
- User wants to fix Redis rate limiter connection timeout.
- Checking existing implementation in src/lib/redis.ts...
- Found `redis = new Redis()` without retryStrategy.
- If Redis is unreachable, this hangs the Node process, violating the 3000ms SLA in AGENTS.md.
- Action Plan:
  1. Add `maxRetriesPerRequest: null` and explicit connectTimeout: 2000.
  2. Implement custom retryStrategy up to 3 attempts.
  3. Run `npm run test:redis` to verify behavior.
```

6.3 识别 AI 陷入的典型"死循环"

在日常开发中，必须对以下两类死循环保持敏锐：

1. 依赖狂躁循环 (The Dependency Loop)

- **特征**：AI 尝试使用一个未经测试的新库，安装报错后，在推理流中尝试更换 3 个不同的版本或换成另一个更陌生的第三方库。
- **信号**：终端连续出现 `npm install --legacy-peer-deps` 超过 2 次。

2. 补丁打地鼠循环 (The Regression Loop)

- **特征**：修改 A 文件导致测试用例 B 挂掉；它去改 B，结果 C 模块报错；它回头去改 C，A 又坏了。
- **信号**：测试通过率在 80% 和 90% 之间反复横跳，且反复修改同一批文件。

6.4 介入三部曲：打断、修正与接管

第一步：果断打断（`Ctrl + C`）

按下 `Ctrl + C`，立即终止当前任务，阻止 Token 损耗。

第二步：点对点纠偏（直接对话）

打断后直接告诉它盲区所在：

```
你刚才试图引入 axios-retry，但本项目严禁使用第三方 HTTP 重试库。请使用原生的 AbortController 实现超时，重新规划。
```

第三步：人手接管与 Git 回滚

如果代码已经被改乱，先检查差异并确认该文件没有需要保留的人工改动，再只恢复目标文件：

```
git diff -- src/lib/redis.ts
git restore -- src/lib/redis.ts
```

在 AGENTS.md 中追加一条红线："严禁引入外部重试依赖"。

6.5 章节实操配套：Codex Web Copilot (Chrome 扩展沙盒样例)

为了让读者直观体验如何使用 Codex 思考链引导与 Anti-Loop 护栏进行浏览器插件开发，本项目配套提供了开箱即用的轻量开源样例：

👉 **源码沙盒目录**：examples/ch06-chrome-extension

核心亮点：

1. **纯原生 Manifest V3**：零打包依赖，直接在 Chrome 浏览器中「加载已解压的扩展程序」即可 1 分钟开箱体验；
2. **严格 CSP 护栏**：在 `AGENTS.md` 中严禁内联脚本与 `eval()`，展示 AI 智能体如何在最严苛的浏览器安全沙盒下编写高可用代码；
3. **自动化测试守卫**：执行 `npm test` 自动验证 MV3 规范与脚本语法。

🚨 翻车自救与避坑速查表

常见踩坑现象	致命原因	排查与处理
AI 陷入自我修改死循环停不下来	提示词缺少明确的重试上限阈值	立即 `Ctrl + C` 中断，在对话中追加指令："已触发重试阈值，停止自旋，输出排查结论"

打断后重新提问，AI 忘记了前面的背景	会话上下文丢失	使用 `codex resume` 恢复原有会话线程，保留先前的思考上下文
AI 改坏了多个历史核心文件	未在修改前隔离改动	先用 `git status` 和 `git diff` 确认要保留的文件；仅恢复确认可丢弃的改动，并为后续任务使用独立分支或工作树

Ch.07 用真实界面验证前端

> ***问题***：*构建和单元测试不能发现所有遮挡、溢出或点击问题。*
>
> ***本章实践***：*在获得相应工具和权限后，检查页面尺寸、交互和截图。*

在传统的 UI 还原度走查中，最耗费产品经理和前端时间的是“像素眼”校对：

“这个按钮好像往左偏了 4 像素。”
“这个弹窗在移动端尺寸下会被软键盘遮挡。”

在客户端提供相应工具且用户授予权限的前提下，Codex 可以操作浏览器或桌面界面。本章以真实页面的截图、点击和尺寸检查为例；具体能力以当前客户端和运行环境为准。[Computer Use 官方指南]

本章教你如何操纵 Computer Use 自动化完成前端 UI 的设计还原。

理解方式：24 小时不知疲倦的“像素质检员”

把 Computer Use 想象成你工位旁雇佣的专职体验质检员：

```
【传统人工走查】 —> 你一手拿 Figma 设计稿，一手拿手机或切换 Chrome 标签页，
                    肉眼眯着看字体大小对不对，手动缩放窗口查断点，改完代码再手动刷一次页面（低效枯燥）。
【Computer Use】 —> 你给质检员下发任务卡：“对照设计稿，走查 /auth/login”。
                    质检员戴上防蓝光眼镜（截屏分析），右手握住鼠标（模拟点击），
                    量出距离相差 16px，直接在编辑器改好 Tailwind 类名，重新刷页面截屏交差。
```

自动检查能帮助定位界面问题；交互质量和最终设计仍需人工判断。

动手实践：先完成这 3 步

用 3 步跑通你的首次 AI 视觉走查：

1. **步骤一：确认 ChatGPT 桌面端权限已授予**
   在 macOS「系统设置 → 隐私与安全性」中，确认已勾选 **ChatGPT** 的「辅助功能」与「屏幕录制」权限。
2. **步骤二：在本地启动待测前端服务**
   在终端启动项目开发服务器（确保能在浏览器打开）：

   ```
   npm run dev
   # 确认 http://localhost:3000 可访问
   ```

3. **步骤三：在代码模式中唤起 `@Chrome` 发送走查任务**
   在 ChatGPT 桌面端 Codex 模式下输入：

   ```
   @Chrome 请打开 http://localhost:3000 登录页，截取主卡片区域，检查是否有超出视口的横向溢出，并调整 padding。
   ```

7.1 安全第一：沙盒边界与屏幕操作白名单

让 AI 操作你的物理屏幕必须建立严密的安全围栏。为防止 AI 因误识别误点个人隐私聊天软件或改动系统文件，必须建立**应用层面的白名单控制**。

1. GUI 权限白名单

在 ChatGPT 桌面客户端中：

- 首次使用 Computer Use 操作某个特定应用（如 Google Chrome）时，会触发系统弹窗询问；
- 可选择 **“Just this once”**（仅本次允许）或 **“Always allow”**（永久放行）；
- 严禁将非必要应用（如微信、邮件、终端本身）加入白名单；
- **出厂硬限制**：Codex 无法自动操作终端、客户端自身或系统级管理员提权密码弹窗（ `sudo` ），这是操作系统级的硬隔离。

2. 坐标定位与视觉识别机制

Computer Use 遵循高可靠的感知闭环：

```
[全屏/视口截屏] —> [GPT-5.6 视觉模型识别] —> [计算目标元素像素坐标 (x:450, y:230)] —> [执行鼠标点击/滚动]
```

结合 2026 年新增的 **Appshots** 能力（快捷双击），可瞬间将前台焦点窗口的视觉截图与文本上下文直接压入当前会话，免去人工截图粘贴。

7.2 视觉驱动的 UI 走查实战：Figma 还原对比

这是最实用的自动化场景：**让 Codex 自主比对设计图与本地网页渲染，自动微调样式。**

🎯 目标 (Goal)

对比本地网页 `/auth/login` 与设计稿截图 `figma_login_mockup.png`，消除视觉间距差异。

🛑 约束 (Constraints)

- 仅允许调整 `src/app/login/page.tsx` 的 Tailwind 工具类。
- 禁止改动原有 DOM 树与语义化标签。

🧪 自动化执行 Specs

在 ChatGPT 客户端中发送：

```
@Chrome 请按以下步骤完成 UI 视觉走查：

# 🎯 Goal
Compare and align browser rendering with figma_login_mockup.png.

# 🛑 Constraints
- Only use Tailwind utility classes in src/app/login/page.tsx.
- Do not change the DOM structure.

# 🚀 Execution Steps
1. Open Google Chrome and navigate to http://localhost:3000/auth/login.
2. Capture screenshot of the login form container.
3. Compare against assets/figma_login_mockup.png and identify padding discrepancies.
4. Update Tailwind classes in src/app/login/page.tsx to match spacing.
5. Reload page and confirm visual alignment within 2% delta.
```

7.3 Codex 自动走查的概念流程（示意）

```
> Running visual review for /auth/login
> Step 1: Opening Google Chrome on http://localhost:3000/auth/login...
> Step 2: Taking screenshot. Saved to /tmp/screenshot_v1.png
> Step 3: Calling vision model for image comparison.
    Analysis: "Login card padding-top is 16px (pt-4), but mockup requires 32px (pt-8). Font size
is text-base, needs text-xl."
> Step 4: Updating src/app/login/page.tsx via apply_patch...
> Step 5: Reloading Chrome tab and taking validation screenshot...
> Step 6: Vision check: "Visual delta is within 1.2% tolerance. Perfectly aligned."
> Task completed successfully.
```

7.4 进阶：Sites 原位预览与响应式多端巡检

借助 2026 ChatGPT 桌面端内置的 **Sites** 功能与移动端断点走查：

```
@Chrome
# 📱 Mobile Viewport Inspection
1. Open Chrome DevTools.
2. Toggle Device Toolbar and select iPhone 15 Pro (393 x 852).
3. Verify the submit button does not drop below the first screen fold.
4. If obscured, reduce hero section padding to keep the CTA button immediately clickable.
```

用不同视口和真实交互检查页面，记录具体的布局与操作问题，再根据结果修改样式。

7.5 跨界延伸：从 Web 走查到 3D 软件自主操作（Blender 联动）

GPT-6 Astra 可在支持的客户端和工具中参与桌面工作流。以下 Blender 操作是本书的实践示例，须检查实际客户端权限与软件状态；不依赖未经证实的 3D 评测数字。[GPT-6 官方指南]

在桌面环境中，开发者甚至可以让智能体自主打开专业 3D 创作软件 **Blender**：

- **视图操作与视口检查**：智能体通过 Computer Use 自动将 Blender 视口切换至 Shading 材质预览模式或 Rendered 渲染模式；
- **排查模型着色与破损**：智能体直接对 3D 视口进行截图比对，检查材质反射是否合理、法线是否翻转；
- **空间联动工作流**：配合 Ch.13 中详述的 **Tripo3D + Blender + Astra** 3D 自动化管线，串起从提示词、几何体生成到 DCC 渲染验证的可检查流程。

🚑 翻车自救与避坑速查表

常见踩坑现象	致命原因	排查与处理
`Computer Use is not supported on this platform`	当前系统非 macOS 或位于未开放区域	确认使用 macOS 客户端，或改用 Headless Puppeteer 纯脚本方案作为替代
`Failed to capture window: Permission denied`	macOS 屏幕录制权限未正确勾选或需要刷新	打开系统设置，重新开关一次 ChatGPT 的「屏幕录制」权限并彻底重启客户端
AI 鼠标在屏幕上乱点，点不到目标按钮	浏览器缩放比例不是 100% 或多屏幕 DPI 换算异常	将 Chrome 缩放比例重置为标准 100%，并将待测窗口放置于主显示器正中

Ch.08 把构建告警和审批接到手机

> ***问题**：离开电脑后，构建失败或待审操作可能无人处理。*
>
> ***本章实践**：搭建告警与人工审批链路，验证通知、身份和操作结果。*

离开电脑时，构建结果和待审批操作仍需要被及时发现。本章先打通告警，再区分自动审查与人工决策。

示例把 GitHub Actions 的失败通知发到飞书，并演示一个本机决策中转网关。`--approve-for-me` 只审查适用的权限请求；生产发布还需独立确认项目、版本与部署结果。

理解方式：告警与审批是两条链路

自动检查负责尽早发现问题；通知负责把结果送到手机；审批则必须由能核对身份和任务的系统处理：

```
GitHub Actions 检查失败 —> 发送飞书告警 —> 人在手机查看构建日志
Codex 权限请求          —> 按当前策略审查 —> 需要时由人确认
生产部署请求            —> 核对项目、版本和审批人 —> 独立部署程序执行并验证
```

这三条链路需要分别配置和验收。收到消息或记录“批准”都不等于部署成功。

动手实践：先完成这 3 步

用 3 步搭建你的移动告警通知通道：

1. **步骤一：创建飞书自定义群机器人** 在群设置中添加机器人，复制其获得的 Webhook URL。
2. **步骤二：在终端发送一条测试告警卡片**
   先在终端设置 `FEISHU_WEBHOOK_URL` 环境变量，再运行 curl 测试通知：

   ```bash
   curl -X POST -H "Content-Type: application/json" \
     -d '{"msg_type":"text","content":{"text":"Codex 测试通知：请确认手机能收到这条消息。"}}' \
     "$FEISHU_WEBHOOK_URL"
   ```

3. **步骤三：在沙盒中运行检查任务** 在已确认测试不会改动生产数据后，可启动受工作区沙盒约束的任务：

   ```bash
   codex exec --sandbox workspace-write --approve-for-me "执行全仓回归测试并重构陈旧类型定义"
   ```

8.1 双层看护分流架构

下图展示建议的分流方式；自动审查是否放行取决于实际权限策略，手机通知不会自动授予部署权限：

```
[项目检查] —(失败)—> [飞书告警] —> [人在手机查看日志]
[Codex 权限请求] —> [当前沙盒与审批策略] —> [人处理未获授权的请求]
[部署请求] —> [身份与任务核对] —> [人工确认] —> [独立部署与结果验证]
```

配套参考开源工程：

- 关联参考工程：plugins-codex-feishu
- 本机极简穿透网关：scripts/codex-watchdog

8.2 实战：GitHub Actions 失败推送与 Webhook 配置

在含有 `npm test` 脚本的 Node.js 项目中，创建 `.github/workflows/codex-watchdog.yml`。先把飞书机器人 URL 配为仓库 Secret `FEISHU_WEBHOOK_URL`。检查失败时发送简短告警，并保留失败状态；具体错误在 Actions 日志中查看：

```
name: Codex Agent Watchdog

on:
  push:
    branches: [ main ]
  workflow_dispatch:

jobs:
  agent-build:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout Code
        uses: actions/checkout@v4

      - name: Setup Node.js
        uses: actions/setup-node@v4
        with:
          node-version: 20

      - name: Run Project Checks
        id: check
        continue-on-error: true
        run: |
          npm ci
          npm test

      - name: Push Fail Notice to Mobile
        if: steps.check.outcome == 'failure'
        env:
          FEISHU_WEBHOOK_URL: ${{ secrets.FEISHU_WEBHOOK_URL }}
        run: |
          curl -fsS -X POST -H 'Content-Type: application/json' \
            -d '{"msg_type":"text","content":{"text":"项目检查失败：请打开 GitHub Actions 查看本次运行日志。"}}' \
            "$FEISHU_WEBHOOK_URL"

      - name: Keep Check Failure Visible
        if: steps.check.outcome == 'failure'
        run: exit 1
```

8.3 户外移动端双向交互与审批

1. 场景：生产环境部署审批

下面是一条需要额外审批系统才能实现的通知示意；前述 Actions 告警和本机中转网关不会自动发送这张卡片：

```
🧯 [Codex Auth Requested]
Project: pmer-cn-saas
Action: Deploy to production (Vercel)
Change summary: Implemented Stripe subscription webhook in /api/stripe.
```

```
Tests: 12 passed, 0 failed.
[回执指令]: 回复 "1" 批准发布，回复 "0" 记录拒绝，后续操作需人工确认。
```

2. 在本机检查审批中转网关

配套 `scripts/codex-watchdog/lib/gateway.js` 只接收决策并写入临时信号文件，**不会直接部署、终止 Codex 或回滚 Git**。它默认只监听 `127.0.0.1`，必须设置随机令牌。要从飞书接入，需要另行配置经身份验证的转发层，并由实际部署程序核对项目、任务和决策时效。

```
cd scripts/codex-watchdog
npm ci
export CODEX_WATCHDOG_TOKEN="$(openssl rand -hex 32)"
node bin/cli.js gateway --port 8080 &
gateway_pid=$!
```

仍在同一终端发送本地测试请求，返回 `denied` 表示已记录拒绝；它不会删除文件。测试后停止网关：

```
curl -fsS http://127.0.0.1:8080/api/mobile-reply \
  -H "Authorization: Bearer $CODEX_WATCHDOG_TOKEN" \
  -H 'Content-Type: application/json' \
  -d '{"userMessage":"0"}'
kill "$gateway_pid"
```

8.4 关联参考工程：飞书助理 (Codex Feishu Sentinel)

可以查看作者维护的 **plugins-codex-feishu（飞书助理）**，了解飞书通知与审批的实现。使用前以该仓库当前 README、所需权限和测试结果为准。

按需要分别核对以下能力：

1. **日常巡检与值班日报（极速上手）**：
   - 核对日报的数据来源、发送对象与调度方式，并先发到测试会话。
2. **移动端警报与双向审批（离线编排）**：
   - 核对告警触发条件、送达情况和移动端回复的接收方式。
   - 手机端可提交审批意见；部署程序还应核对身份、项目、任务和有效期。拒绝操作只记录决定，不自动回滚代码。
3. **沉淀与进阶知识库（团队协同）**：
   - 如需写入飞书文档或多维表格，先核对应用权限与实际写入结果。

> 💡 ***配置前核对**：若参考工程提供飞书应用清单，仍需在飞书开放平台检查所需权限、事件订阅和回调连接，再在测试群验证告警。*

🚦 翻车自救与避坑速查表

常见踩坑现象	致命原因	排查与处理
手机接收不到 Webhook 消息	机器人安全设置中未配置自定义关键词或 IP 白名单	检查飞书机器人设置中的“安全设置”，设置包含关键词（如 Codex）或签名校验
手机回复了“1”但没有部署	本机网关只记录决策，没有部署程序	检查经过鉴权的转发层、任务核对和独立部署程序
手机反复收到审批消息	同一请求被重复发送	为审批请求设置唯一 ID、去重和过期时间

8.5 把通知与确认留给人

把重复检查交给 CI，把失败信息及时送达手机，可以减少反复守在电脑前等待的时间。对发布、数据变更等操作，仍要让有权限的人核对具体任务，并检查执行结果。

Ch.09 先建立基线，再逐步改造旧系统

> ***问题***：*无测试的旧系统难以判断改动是否破坏现有行为。*
>
> ***本章实践***：*先画依赖图并补关键路径测试，再按小范围提交重构。*

在独立开发或承接存量项目时，最让人胆战心惊的莫过于接手前人留下、毫无文档且零单测覆盖的“屎山”工程。任何微小的改动，都可能引爆隐藏在深处的连环地雷。

面对这类遗留系统，千万不要冲动推倒重写。先读清入口、依赖和关键行为，再用可重复的测试保护现有功能；模型上下文大小不能代替逐步验证。

理解方式：在高速公路上给飞驰的老卡车换轮胎

重构正在跑着的遗留老系统，就像这样一种高危操作：

【莽撞全面重写】 —> ❌ “这台老卡车太破了，我要在路边重新手搓一台新车！”
（结果新车造了 6 个月没造出来，业务早断粮饿死了。）
【渐进式微创解耦】—> ✅ 老卡车继续在高速上拉货（核心业务不中断）：
1. 先拿无人机拍底盘（让 AI 扫描全仓生成拓扑地图）；
2. 装上安全防侧翻支架（为关键接口编写基准行为快照）；
3. 每次只拆卸一颗生锈的螺丝，换好测跑没问题再动下一颗。

Codex 就是带毫米级激光仪的特种维修臂，只要你给它戴上测试护栏，它就能精准切下坏疽而不伤及健康肌肉。

动手实践：先完成这 3 步

接手一个陌生混乱仓库的第一天，按这 3 步稳健破局：

1. **步骤一：让 AI 输出全仓拓扑地图（只读探索）**
   在终端运行只读沙盒分析，生成 Mermaid 依赖图：

   ```
   codex exec --sandbox read-only "分析项目整体架构与数据库模型，在 docs/topology.md 中输出 Mermaid 依赖图"
   ```

2. **步骤二：为要修改的接口拍一张“行为快照”（录制基准）**
   要求 Codex：“不要动 `src/api/` 的业务代码，只为现有接口补齐 3 个成功与失败的单元测试”。
3. **步骤三：单点切除并用测试断言守门**
   下发微创重构 Specs，要求：“每次仅抽离一个函数，且修改后自动执行 `npm test`，测试必须全绿”。

9.1 第一步：全景逆向，建立代码拓扑图

接手混乱项目的首要任务是**画出全景地图**。利用 GPT-5.6 Terra 的长上下文扫描能力直接逆向出 Mermaid 实体关系图（ERD）与路由拓扑：

```
# 🎯 Goal
分析当前项目的数据库配置与路由层，生成核心表结构外键关联的 Mermaid ERD 图。
```

```
# 🛑 Constraints
- 只分析 prisma/schema.prisma 或 src/db/models/。
- 排除临时缓存表与第三方集成日志表。

# 🧪 Validation Specs
- 在 docs/database_topology.md 中输出语法正确的 Mermaid 渲染图。
```

Codex 几秒钟即可梳理出核心实体之间的多对多与级联关联，效率远胜人肉翻代码。

9.2 第二步：安全红线，编写行为基准测试

重构遗留代码的绝对铁律是：**动手术前，必须先拉起生命体征监护仪（基准测试）。**

```
# 🎯 Goal
为 src/pages/api/checkout.ts 接口编写基准单元测试，捕获当前真实的入参与返回快照。

# 🛑 Constraints
- 严禁修改 src/pages/api/checkout.ts 本身的任何逻辑代码。
- 使用本地 Jest / Vitest 执行测试套件。

# 🧪 Validation Specs
- 覆盖三种核心分支：正常结算、库存告罄报错、未认证拦截。
- 确保基准测试在当前旧代码上 100% 通过。
```

9.3 第三步：微创手术，实施渐进式解耦

1. 拆解胖控制器 (Fat Controllers)

将混杂在几百行单文件里的鉴权、打折、库存扣减逐个剔除为纯函数：

```
# 🎯 Goal
将 src/pages/api/checkout.ts 中的“折扣计算逻辑”抽离为独立服务 src/services/discountService.ts。

# 🛑 Constraints
- 外部 API 接口的 Request / Response JSON 格式必须保持严格向下兼容。
- 严禁影响 checkout.ts 的其他核心逻辑分支。

# 🧪 Validation Specs
- 运行 npm run test，确保之前编写的结算基准快照测试 100% 全绿通过。
- 运行 npm run lint 无任何类型错误。
```

2. 局部验证与安全回滚

要求 Codex 在每次小范围修改后运行验证命令。若测试失败，先查看差异；确认文件中没有需要保留的人工改动后，再恢复该文件：

```
git diff -- src/pages/api/checkout.ts
git restore -- src/pages/api/checkout.ts
```

🚑 翻车自救与避坑速查表

常见踩坑现象	致命原因	排查与处理

一句话让 AI “重构整个项目”，导致系统彻底瘫痪	步子迈太大，AI 在大规模文件修改中出现幻觉与类型断裂	严格约束重构粒度，每次 Prompt 仅允许改动 1 个特定服务或函数
重构后旧功能莫名其妙丢失或报错	缺少重构前的行为基准快照测试	坚持“测试先行”，在修改前必须先让 AI 针对旧代码写出 100% 通过的基准单测
AI 在重构中擅自把底层框架给升级了	约束中未锁死 package.json	在 AGENTS.md 与 Constraints 中强调：“严禁变更 package.json 内的任何依赖版本”

Ch.10 验证订阅制 SaaS 的支付闭环

> ***问题***：*订阅产品需要同时验证登录、支付回调和权限状态。*
>
> ***本章实践***：*用配套 Next.js 样例跑通 Stripe 测试支付与 Webhook 验签。*

订阅产品的第一个工程里程碑，是在测试环境里验证注册、支付回调和权限变更能形成闭环。真实收款还需要部署、风控和合规检查。

本章使用 `Next.js 15 (App Router) + Supabase (PostgreSQL) + Stripe` 配套工程，在本地跑通订阅支付测试。完成时间取决于账号、环境和现有代码。

> 📦 ***配套实战源码***：*examples/ch10-saas-mvp —— 完整可运行的订阅制 AI 翻译工具（TransFlow），自带 CAP 协议 `AGENTS.md`，`npm install && npm run build` 已验证通过。*

理解方式：开出能收银的“流动煎饼摊”

很多人做 SaaS 产品，总想着造一座五星级大酒店：

```
【空想五星级酒店】 —> 花 6 个月设计华丽大堂、采购高档地毯、招几十个服务员（过度工程），
                       结果开业第一天发现没人愿意来吃饭，直接倒闭。
【流动煎饼果子小推车】—> ✅ 你只需要 3 件核心装备：
                       1. 煎饼炉子（Next.js 核心业务页面：提供核心价值）；
                       2. 储物箱（Supabase / PostgreSQL：存用户和订单数据）；
                       3. 扫码收款码（Stripe 订阅：能把钱结结实实收进账户）。
```

只要客人扫码付了 10 块钱，你把热腾腾的煎饼交到他手上，商业闭环就算真正跑通了！

动手实践：先完成这 3 步

用 3 步快速打通本地 Stripe 支付联调闭环：

1. **步骤一：获取 Stripe 测试密钥并在本地声明**
   登录 Stripe 开发者控制台，获取测试公私钥并写入 `.env.local`：

   ```
   STRIPE_SECRET_KEY="sk_test_..."
   NEXT_PUBLIC_STRIPE_PUBLISHABLE_KEY="pk_test_..."
   ```

2. **步骤二：启动 Stripe CLI 本地 Webhook 转发**
   在终端运行监听命令，打通本地隧道：

   ```
   stripe listen --forward-to localhost:3000/api/webhooks/stripe
   # 终端将输出专属签名秘钥：whsec_...
   ```

3. **步骤三：让 Codex 沙盒执行并验证 Webhook 签名逻辑**
   将获得的 `whsec_...` 注入环境，并命令 Codex："编写测试用例，模拟触发 `checkout.session.completed`，必须返回 200 且用户权限状态变更为 ACTIVE"。

10.1 初始化骨架与数据库 Schema 设计

我们的目标是做一款订阅制的 AI 翻译工具。首先使用 Next.js 初始化项目，并配置好基础约束。接着，让 Codex 生成核心的数据库模型。

1. 编写 Prisma Schema (Prisma 实体建模)

向 Codex 下达目标 Specs：

```
# 🎯 Goal
编写 Prisma 数据库模型，支持用户表（User）、订阅表（Subscription）与翻译记录表（TranslationRecord）。

# 🛑 Constraints
- 数据库驱动使用 PostgreSQL（连接 Supabase）。
- 订阅状态必须是 Enum 类型，包含 ACTIVE, CANCELED, EXPIRED。

# 🧪 Validation Specs
- 运行 `npx prisma validate` 无错误提示。
```

Codex 会自动输出标准的 `prisma/schema.prisma` 模型，包含级联删除与外键索引：

```
datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL")
}

generator client {
  provider = "prisma-client-js"
}

enum SubscriptionStatus {
  ACTIVE
  CANCELED
  EXPIRED
}

model User {
  id             String               @id @default(uuid())
  email          String               @unique
  createdAt      DateTime             @default(now())
  subscription   Subscription?
  records        TranslationRecord[]
}

model Subscription {
  id             String               @id @default(uuid())
  userId         String               @unique
  stripeSubId    String               @unique
  status         SubscriptionStatus
  priceId        String
  currentPeriodEnd DateTime
```

```
  user           User                @relation(fields: [userId], references: [id], onDelete:
Cascade)
}

model TranslationRecord {
  id             String              @id @default(uuid())
  userId         String
  sourceText     String
  translatedText String
  createdAt      DateTime            @default(now())
  user           User                @relation(fields: [userId], references: [id], onDelete:
Cascade)
}
```

10.2 集成 Stripe 订阅与 Webhook 监听

支付系统的核心是**回调安全**。当用户支付成功后，Stripe 的服务器会向你的 Next.js 服务发送一个 Webhook 请求。

自动化 Specs：向 Codex 下达路由编写指令

```
# 🎯 Goal
实现 Next.js 15 App Router 风格的 Stripe Webhook 路由处理程序 `/api/webhooks/stripe`。

# 🛑 Constraints
- 必须使用 `stripe.webhooks.constructEvent` 验证请求签名的真实性，防止伪造攻击。
- 当接收到 `checkout.session.completed` 或 `invoice.payment_succeeded` 事件时，更新用户的订阅状态。
- 严禁对响应体进行普通 JSON 解析，Stripe 验证要求使用原始 req.text() 字符串。

# 🧪 Validation Specs
- 编写单元测试，验证签名失败时返回 400，成功时返回 200。
- 真实订阅周期必须从 Stripe 响应中读取，不得硬编码。
```

Codex 生成的标准实现：

```
// File: src/app/api/webhooks/stripe/route.ts
import { NextResponse } from 'next/server';
import Stripe from 'stripe';
import { prisma } from '@/lib/prisma';

const stripe = new Stripe(process.env.STRIPE_SECRET_KEY!, {
  apiVersion: '2026-04-22.dahlia' as any,
});

export async function POST(req: Request) {
  const body = await req.text();
  const signature = req.headers.get('stripe-signature')!;

  let event: Stripe.Event;

  try {
    event = stripe.webhooks.constructEvent(
      body,
      signature,
      process.env.STRIPE_WEBHOOK_SECRET!
```

```
    );
  } catch (err: any) {
    return NextResponse.json({ error: `Webhook Error: ${err.message}` }, { status: 400 });
  }

  if (event.type === 'checkout.session.completed') {
    const session = event.data.object as Stripe.Checkout.Session;
    const stripeSubId = session.subscription as string;
    const customerEmail = session.customer_details?.email!;

    const subscription = await stripe.subscriptions.retrieve(stripeSubId);

    await prisma.subscription.upsert({
      where: { stripeSubId },
      update: {
        status: 'ACTIVE',
        currentPeriodEnd: new Date(subscription.current_period_end * 1000),
      },
      create: {
        stripeSubId,
        status: 'ACTIVE',
        priceId: subscription.items.data[0]?.price.id || 'default',
        currentPeriodEnd: new Date(subscription.current_period_end * 1000),
        user: { connect: { email: customerEmail } }
      }
    });
  }

  return NextResponse.json({ received: true });
}
```

10.3 本地联调：用 Stripe CLI 跑通支付闭环

1. 在本地运行 Stripe 转发：

```
stripe listen --forward-to localhost:3000/api/webhooks/stripe
```

2. 触发一次真实的结账测试：

```
stripe trigger checkout.session.completed
```

3. 观察终端是否输出 `200 OK`，并使用 Prisma Studio 查验数据库：

```
npx prisma studio
```

商业 MVP 的价值在于上线速度。用最严密的边界规则约束 AI，换取最极致的交付体验。

翻车自救与避坑速查表

常见踩坑现象	致命原因	排查与处理
`Webhook Error: No signatures found matching the expected signature`	使用了 `await req.json()` 解析 Body，破坏了加密签名原貌	严格使用 `await req.text()` 获取 Raw String，再传入 `constructEvent`

测试信用卡支付成功后，数据库无任何新增记录	Webhook URL 配置错误或本地没有运行 `stripe listen` 转发	启动 `stripe listen`，确认监听终端打出 `200 OK [POST /api/webhooks/stripe]`
`DATABASE_URL` 连不上 Supabase	忘记开启 Supabase 连接池（Session / Transaction 端口混淆）	检查数据库连接串，确保在 Serverless 环境使用 6543 连接池端口

Ch.11 用 Expo 构建并检查移动应用

> ***问题***：*移动端依赖、设备调试和签名构建容易出现环境差异。*
>
> ***本章实践***：*运行配套 Expo 工程，在真机检查界面，并按平台要求配置 EAS 构建。*

做完网页版 SaaS 后，很多独立开发者希望能将触角延伸到移动端。但在传统的原生开发（React Native / Flutter）中，最耗费时间的往往是复杂的本地开发环境配置：iOS 证书管理、Android Gradle 报错、Cocoapods 冲突，这些环境地狱常常让人望而却步。

EAS Build 可以减少本地构建环境的要求，但 iOS 签名、开发者账号和上架审核仍需配置。先在设备上验证功能，再分别生成开发和生产构建。Expo 构建准备

> 📦 ***配套实战源码***：*examples/ch11-expo-mobile —— 完整可运行的 Expo SDK 57 工程（Expo Router `src/app` 路由 + NativeWind + EAS 三档打包配置），自带 CAP 协议 `AGENTS.md`，已通过 `npx expo lint`（零错误）与 `npx expo-doctor`（20/20）验证。*

理解方式：写好剧本，让“云端工坊”为你定制戏服

跨端开发不需要把你的电脑变成重型加工厂：

```
【传统原生开发】 —> 你在家里自己买炼钢炉（装几十 G 的 Xcode 和 Android Studio）、
                    买织布机（配各种 Gradle/Cocoapods 运行环境），动不动断电报错，三天造不出一只鞋。
【Expo + EAS 模式】—> ✅ 你只负责写剧本（写 React Native / TypeScript 核心代码）：
                    – 剧本写好后，一键发给「云端特种裁缝工坊」（EAS Build）；
                    – 云端自动为你剪裁出 iOS 的 IPA 和 Android 的 APK 安装包；
                    – 手机直接扫码就能把做好的衣服套在身上跑！
```

Codex 就是你在工坊里的剧本精修助理，负责在你写漏了依赖或样式语法时，自动对齐版本。

动手实践：先完成这 3 步

用 3 步在手机真机上看到你的第一个跨端 App：

1. **步骤一：手机下载安装 Expo Go 客户端**
   在 App Store 或各大安卓应用商店搜索并安装「Expo Go」。
2. **步骤二：在项目终端启动本地开发服务器**
   进入项目目录，运行开发命令：

   ```
   npx expo start
   # 终端将打出巨大的二维码矩阵
   ```

3. **步骤三：用手机相机扫码即可秒开实时热更新**
   使用与当前 SDK 兼容的 Expo Go 或开发构建扫码打开项目。代码变更通常会触发刷新；原生模块变更可能需要重新构建。

11.1 Expo 项目极速初始化与规范设定

为了能使用 EAS 进行云端免配置打包，我们首先初始化一个标准的 Expo 项目。

1. 编写 App 初始化 Specs

给 Codex 下达 Specs 指令：

```
# 🎯 Goal
初始化一个使用 TypeScript 的 React Native Expo 项目。

# 🛑 Constraints
- 使用最新稳定的 Expo SDK 57，搭配 Expo Router 实现基于文件系统的路由。
- 引入 NativeWind 作为 Tailwind 风格的样式方案。
- 目录约定使用 src/ 前缀（即 src/app/ 作为路由根）。

# 🧪 Validation Specs
- 运行 `npx expo lint` 无错误。
- 运行 `npx expo-doctor` 检查依赖版本对齐（必须 20/20 通过）。
```

目录结构规范：

```
src/
├── app/
│   ├── index.tsx          # APP 首页
│   └── _layout.tsx        # 全局路由导航布局
├── components/
└── hooks/
app.json                   # Expo 核心配置文件
package.json
```

11.2 解决移动端顽疾：原生依赖冲突

React Native 开发最忌讳使用普通 `npm install` 安装带底层原生代码的第三方包（如相机、传感器、手势库）。

铁律：使用 `npx expo install` 对齐版本

如果依赖版本不匹配，在 TUI 中输入纠偏指令：

```
请改用 `npx expo install react-native-reanimated` 重新安装该依赖，它会自动适配当前的 Expo SDK 版本。在本项目中，严禁使用普通的 `npm install` 安装任何原生依赖包。请将该铁律写入 AGENTS.md。
```

> 💡 ***主理人心法：*** *`npx expo install` 会选择与当前 Expo SDK 兼容的依赖版本。原生模块报错时先运行 `npx expo-doctor`，再核对依赖、构建日志和平台限制。*

11.3 EAS Build 云端打包与证书自动化

在传统流程中，申请苹果开发者证书往往会卡住新手几天。现在通过 EAS，全流程可以在云端自动化解决。

1. 配置 `eas.json`

```
{
  "cli": {
    "version": ">= 9.0.0"
  },
```

```
  "build": {
    "development": {
      "developmentClient": true,
      "distribution": "internal"
    },
    "preview": {
      "distribution": "internal"
    },
    "production": {
      "ios": {
        "simulator": false
      }
    }
  }
}
```

2. 让 Codex 监考云端构建日志

```
# 启动 EAS iOS 打包任务，把日志写入本地
eas build --platform ios --profile production --non-interactive 2>&1 | tee eas-build.log
```

如果打包遭遇异常，直接将日志转交给 Codex 分析：

```
codex exec --sandbox read-only "分析 eas-build.log，定位失败原因并给出修复命令"
```

最终 EAS 会返回一个安装二维码，你只需用手机扫码即可直接安装测试版 App。

翻车自救与避坑速查表

常见踩坑现象	致命原因	排查与处理
手机扫码显示“Could not connect to development server”	手机与电脑不在同一个 Wi-Fi 局域网，或电脑开启了路由器防火墙	运行 `npx expo start --tunnel`，强制使用公网穿透隧道模式，零网络门槛
`Invariant Violation: "main" has not been registered`	入口路由文件丢失或 `app.json` 中配置的 entry 路径错误	检查 `package.json` 中的 `"main": "expo-router/entry"` 是否被意外篡改
安装新组件后终端直接报红红屏	误用了 `npm install` 安装了不兼容当前 SDK 版本的包	运行 `npx expo install <package-name>`，或运行 `npx expo-doctor` 诊断修复

Ch.12 从产品使用数据到获客实验

> ***问题***：*功能上线后仍需要知道用户是否完成关键流程及从哪里流失。*
>
> ***本章实践***：*定义少量业务指标，建立日报，并用可回滚的小实验验证渠道。*

在“实战产品说”微信公众号和 pmer.cn 上，我写过很多关于“独立开发与副业变现”的文章。很多开发者最容易走入的死胡同是：**持续优化实现细节，却没有验证真实用户是否需要这个功能。**

代码写得再优雅、架构配置得再完美，只要没人用，它就是一个精美的摆设。在 AI 原生时代，我们不仅要让 Codex 帮我们“生产产品”，更要让它帮我们“转动商业轮盘”。

理解方式：造一台日夜自转的“水利灌溉水车”

商业飞轮不是靠你每天手动人肉挑水：

```
【人力挑水型创业】 —> 你每天写代码到半夜，第二天还要四处发传单拉客户，
                       只要你生病歇一天，产品断更、客户归零（极其脆弱）。
【自动水车商业飞轮】—> ✅ 你在小河边组装好这台水利水车：
                       1. 导流槽（SEO 与自动化 Sitemap：持续吸附长尾搜索引擎流量）；
                       2. 碾米轮叶（核心 SaaS 功能：为用户解决实际问题）；
                       3. 出米漏斗（Stripe 自动结账与订阅扣款：持续产生现金流）；
                       4. 水位仪表盘（每日飞书数据战报：监控转化率与活跃指标）。
```

一旦把这套流水线组装完毕，无论你是在睡觉、吃饭还是旅行，水车日夜自转，源源不断为你创造价值。

动手实践：先完成这 3 步

用 3 步迈出商业化闭环的第一步：

1. **步骤一：部署自动化 Sitemap 生成脚本**
   在项目部署流水线中自动执行 `scripts/generate-sitemap.js`，确保每次新增 Markdown 博客或页面自动被 Google 收录。
2. **步骤二：挂载每日早 8 点营收推送**
   配置定时脚本，每天准时往手机推送新增注册与付费活跃订阅数，培养对商业数字的敏感度。
3. **步骤三：在社区抛出你的“一句话价值提案”**
   在 X (Twitter)、即刻、V2EX 或微信朋友圈发布你的产品 MVP 链接，验证首批种子用户的真实付费反馈。

12.1 “一人 SaaS 公司”的自动化流量管道

一个健康的独立项目，其流量获取（SEO、长尾关键词、社交媒体）应该和它的代码库一样，是一套自动运转的流水线。

实战：让 Codex 自主维护 SEO 博客与 Sitemap

```
// File: scripts/generate-sitemap.js
const fs = require('fs');
const path = require('path');

const BASE_URL = 'https://pmer.cn';
const blogDir = path.join(__dirname, '../content/blog');

function getBlogSlugs() {
  if (!fs.existsSync(blogDir)) return [];
  return fs.readdirSync(blogDir)
    .filter(file => file.endsWith('.md'))
    .map(file => `/blog/${file.replace('.md', '')}`);
}

function generate() {
  const staticPages = ['/', '/auth/login', '/features'];
  const blogPages = getBlogSlugs();
  const allUrls = [...staticPages, ...blogPages];

  const xml = `<?xml version="1.0" encoding="UTF-8"?>
<urlset xmlns="http://www.sitemaps.org/schemas/sitemap/0.9">
  ${allUrls.map(url => `
```

```
  <url>
    <loc>${BASE_URL}${url}</loc>
    <changefreq>daily</changefreq>
    <priority>${url === '/' ? '1.0' : '0.8'}</priority>
  </url>`).join('')}
</urlset>`;

  fs.writeFileSync(path.join(__dirname, '../public/sitemap.xml'), xml);
  console.log('✅ Sitemap.xml generated successfully!');
}

generate();
```

12.2 对接业务数据，获取每日商业战报

为了让你对钱的动向足够敏感，每天早上拉取 Stripe 收益和用户增长，直接通过 Webhook 发送到手机：

```
// File: scripts/daily-report.js
const { PrismaClient } = require('@prisma/client');
const prisma = new PrismaClient();
const axios = require('axios');

async function sendReport() {
  const yesterday = new Date(Date.now() - 24 * 60 * 60 * 1000);
  const userCount = await prisma.user.count({
    where: { createdAt: { gte: yesterday } }
  });

  const activeSubs = await prisma.subscription.count({
    where: { status: 'ACTIVE' }
  });

  const reportText = `📊 【实战产品战报】\n昨日新增注册用户: ${userCount} 人\n当前总活跃订阅:
${activeSubs} 个\n—— 继续加油！`;

  await axios.post(process.env.MOBILE_WEBHOOK_URL, {
    msg_type: 'text',
    content: { text: reportText }
  });
}

sendReport();
```

12.3 代码更容易生成后，哪些能力仍重要

当代码生成速度提高时，需求判断、用户触达和服务质量仍决定产品能否被持续使用。对独立开发者和产品经理，重要的能力包括：

1. **理解用户实际要解决的问题。**
2. **了解所在领域的约束和流程。**
3. **用证据验证产品和商业假设。**

🚦 翻车自救与避坑速查表

常见踩坑现象	致命原因	排查与处理
产品上线后连续 1 个月零访问	闭门造车，未在早期接入 SEO 与社媒种子流量	立即在 Twitter/小红书/社区发帖分享产品开发故事，并用脚本输出多语种长尾博文
沉迷于重构优化性能，迟迟不敢发推	完美主义陷阱，恐惧被外界挑刺	记住：只要核心收单流程没坏，其他小瑕疵直接发，在真实用户反馈中迭代
为了微小的样式调整浪费大半天	没有把视觉微调全权委托给 Codex	用 Ch.07 学习的 Computer Use 自动巡检，把时间抢回来去跟付费用户聊天

Ch.13 核对 2026 年的模型与工具变更

> ***问题***：*模型、客户端和命令更新频繁，旧版本说明可能已失效。*
>
> ***本章实践***：*按本书资料截止日核对官方变更，运行兼容性检查后再迁移。*

前面十二章建立的，是一套不依赖具体版本的编排方法论。但方法论要落地，就必须踩在真实的工具地面上。本次核对覆盖 9 月 24 日之后的模型选型、CLI 设置、桌面协作与用量控制变化，并给出具体迁移命令及适用范围。

理解方式：超音速客机的新一代航电换装

不要对频繁的版本迭代感到恐慌。用飞行员的视角来理解这次升级：

```
【驾驶心智】 ──> 你的飞行手册没有变：依然是目标驱动（起飞与降落目标）、边界约束（航线与安全高度）。
【动力引擎】 ──> 动力系统换装为前沿旗舰「GPT-6 Astra」（具备超视距雷达与全自主航电控制的顶级核心）、
                 日常复杂编码发动机「GPT-6.1 Sol」以及明确小任务的巡航发动机「GPT-6 Luna」。
【驾驶舱屏】 ──> Worktree 终端、审查导航与手机审批继续改善协作体验；
                 能力是否开放，仍需核对已安装客户端与工作区设置。
【自动副驾】 ──> 旧的 --full-auto 兼容入口已弃用；新调用显式设置沙盒与审批模式。
```

只要心智在握，新工具只会让你飞得更快、更稳。

动手实践：先完成这 3 步

用 3 步完成你的 2026 工具链体检与迁移：

1. **步骤一：扫描项目和配置中的旧模型代号** 在项目根目录运行 grep 扫描：

   ```
   grep -rn "gpt-5.4" ~/.codex/config.toml .
   ```

2. **步骤二：检查本机 CLI，按需更新** 先核对当前版本；需要更新时再运行 npm 全局更新命令：

   ```
   codex --version
   npm install -g @openai/codex@latest
   codex --version
   ```

3. **步骤三：迁移已弃用的 `--full-auto` 调用** 在所有本地别名（aliases）与 CI 脚本中，将 `--full-auto` 替换为：

   ```
   codex exec --sandbox workspace-write "Review the current changes and report issues"
   ```

13.1 桌面端与移动端：影响日常协作的变化

在当前桌面客户端进入 Codex，并分别核对账号与工作区权限。旧版把"7 月合并"推导成所有账号都能获得全部能力，依据不足，已改为按实际入口与权限核对。

- **Worktree 与任务引导**：9 月 29 日的 App 26.928 改善已挂载 worktree 的终端启动，并在追加指令时保留此前回答。后续版本改进批量移动聊天、审查导航和 Windows / WSL 恢复。官方更新日志
- **输入建议（Composer predictions）**：10 月 9 日起提供 beta。Tab 接受的是可编辑草稿，发送仍是另一个动作；适用于受支持地区、18 岁以上的个人 Pro 用户，在 Astra 或 GPT-6.1 Sol 的本地与 SSH 聊天中使用。开关位于 Settings → General → Composer。输入建议说明
- **手机接续**：近期 iOS 更新改进审批展示、加号菜单中的插件入口与排队任务恢复。手机审批前仍需确认目标电脑、任务及操作内容。移动端更新日志

```
# 挂载或交接聊天前，检查实际仓库与 worktree。
git status --short
git worktree list
codex --version
```

各客户端支持的登录方式与功能范围分别核对；API Key 与 ChatGPT 订阅的访问、计费规则不同。Codex 模型说明

13.2 GPT-6 系列：加入 GPT-6.1 Sol 后如何选型

截至 **2026 年 10 月 21 日**，9 月 29 日推出的 **GPT-6.1 Sol** 是复杂 Codex 工作的当前起点；最难的任务继续评估 Astra，明确小任务使用 Luna。GPT-6 Sol 和 GPT-5.6 Sol / Terra / Luna 在推出期间仍可用。GPT-6.1 Sol 首发覆盖 Plus、Pro、Business、Enterprise、Edu，后两者需管理员启用；Free、Go 不在首发范围。Codex 模型说明

模型 ID	适合的工作	`reasoning.effort`	边界
`gpt-6-astra`	最复杂的跨代码、浏览器和桌面工具任务	`low`、`medium`、`high`、`xhigh`、`max`	不支持 `none`；Computer Use 仍由客户端和工具执行操作
`gpt-6.1-sol`	日常复杂编码、跨应用和文档的持续任务	`low`、`medium`、`high`、`xhigh`、`max`	当前示例默认值；不支持 `none`、`minimal`
`gpt-6-sol`	日常复杂编码、代码审查和多步智能体工作	`none`、`low`、`medium`、`high`、`xhigh`、`max`	上一代 Sol；保留用于配置迁移对照
`gpt-6-luna`	范围明确的修订、提取、摘要和高频任务	`none`、`low`、`medium`、`high`、`xhigh`、`max`	任务复杂度超出能力时改用 Sol 或 Astra

这些模型支持文本和图片输入、文本输出；API 模型目录列出 **1,050,000 token 上下文窗口**和 **128,000 token 最大输出**。工具调用优先使用 Responses API，尤其 Astra 带工具调用时必须使用 Responses。API 模型目录 · GPT-6 迁移指南

GPT-6.1 Sol 迁移重点：API 默认推理档位为 `medium`；工具调用使用 Responses，Chat Completions 支持该模型但不支持其工具调用。Codex 界面中的 Max / Ultra 依设置开放；Ultra 通过子智能体并行工作，不是 API 的 `reasoning.effort` 值。API 模型说明 · Codex 推理设置

API 计费与 Codex 用量分开看。 下表是截至本次核对时，标准处理、输入不超过 272K token 时每百万文本 token 的 API 标价；它不是 ChatGPT 订阅价，也不是 Codex 额度换算。长上下文、缓存及其他处理档位另按官方定价页核算。

API 模型	输入	缓存输入	输出
`gpt-6-astra`	$10	$1	$50
`gpt-6.1-sol`	$2	$0.10	$10

`gpt-6-sol`	$2	$0.20	$10
`gpt-6-luna`	$0.10	$0.01	$0.50

GPT-6.1 Sol 在标准处理、输入不超过 272K 时，缓存写入另计 $2.50 / 百万 token；长输入与速度档位会改变价格。模型价格

速度单独设置。 10 月 8 日起，GPT-6.1 Sol Ultrafast 向 Pro $500 与符合条件的 Enterprise / Edu 开放，Enterprise 默认关闭，需管理员启用。订阅内用量按 Standard 的 8 倍消耗，购买的积分及企业按量用量按 6 倍计；API Key 使用 API 价格。Sol Ultrafast 支持美国及 EEA＋瑞士的推理数据驻留。先用 Standard，在熟悉任务上比较时延、质量与用量，再决定是否开启。速度与计费说明

选择模型：

```
codex --version
codex --model gpt-6.1-sol
codex exec -m gpt-6-luna "Review the current changes"
```

交互会话也可用 `/model` 切换。若新模型尚未出现在选择器中，先核对客户端和账号可用性，不要把 API 可调用等同于 Codex 登录态可用。Codex 模型说明

本地默认配置示例：

```
# ~/.codex/config.toml
model = "gpt-6.1-sol"
```

此示例只设置本地默认模型。自动审批 `--approve-for-me` 是权限模式，不应声称固定由 Luna 执行。

已证实的生命周期节点

日期	范围	官方记录
2026-03-26	API	`gpt-4-0314`、`gpt-4-1106-preview`、`gpt-4-0125-preview` 停用
2026-05-12	API	`dall-e-2`、`dall-e-3` 停用
2026-08-26	API	Assistants API 停用，迁移至 Responses API 和 Conversations API
2026-08-31	Codex 的 ChatGPT 登录态	`gpt-5.4`、`gpt-5.4-mini` 退役
2026-09-03	API	GPT-6 Astra 发布
2026-09-22	API / Codex	GPT-6 Sol、Luna 发布并开始向 Codex 用户推出
2026-09-29	API / Codex / Work	GPT-6.1 Sol 推出
2026-10-14	Codex 的 ChatGPT 登录态	GPT-5.5 已退役；API 与 API Key 会话不受影响

API 退役日期与官方建议替代模型见 API 退役公告；Codex 登录态的日期与适用范围见 Codex 模型说明和更新日志。不要把某个界面的退役扩大描述成 API 全面下线。

13.3 Codex CLI：以 0.163.0 为本次核对点

截至本书 **2026 年 10 月 21 日**核对时，官方更新日志列出的 CLI 为 10 月 15 日发布的 `0.163.0`。这是有日期的核对结果，本机安装版本可能不同。更新日志

`0.159.1` 已将 GPT-6.1 Sol 加入内置目录。`0.163.0` 提供 `/config` 设置搜索与重启提示、`/keymap` 前缀快捷键配置、`/daemon` 后台服务器管理；追加消息可即时引导任务，恢复聊天时按当前策略恢复已保存权限。重启服务器前看清可能中断的工作。

```
codex --version
npm install -g @openai/codex@latest
codex --version
```

1. 迁移 `--full-auto` 的旧调用

`codex exec --full-auto` 是**已弃用的兼容入口**，当前会提示警告；不应写成“已移除并立即报错”。新的非交互调用明确指定沙盒：

```
codex exec --sandbox workspace-write "Review the current changes and report issues"
```

沙盒限定工作区访问，`--approve-for-me` 则把符合条件的越界请求送交自动审查；二者用途不同。[审批与安全说明]

2. Hooks 使用受支持的事件结构

`config.toml` 的 Hooks 使用事件数组及处理器数组，不能写成 `[hooks] stop = "..."`。以下示例在任务结束时检查当前仓库的空白错误；首次启用时通过 `/hooks` 检查和信任来源。[Hooks 文档]

```
# ~/.codex/config.toml
[[hooks.Stop]]

[[hooks.Stop.hooks]]
type = "command"
command = "git diff --check"
```

3. 插件以当前目录中的 ID 安装

```
codex plugin list
codex plugin add --help
```

`codex plugin add` 接受目录中显示的 `PLUGIN@MARKETPLACE` 形式；`codex plugin install` 不是当前 CLI 子命令。安装前先从目录核对插件 ID 和来源。[Codex 更新日志]

4. 自动审批不绑定公开模型

`--approve-for-me` 将适用的审批请求交给自动审查。它不扩大沙盒权限，也不表示所有高风险操作一定获准，更不能推断后台固定使用 GPT-6 Luna。[权限说明]

```
codex --approve-for-me "Review the current changes and suggest fixes"
```

13.4 Skills 生态：把重复流程沉淀为资产

如果说 AGENTS.md 是项目的“通用宪法”，**Skills 就是某一类任务的“专项工艺流程”**：

```
skills/
└── pr-review/
    └── SKILL.md      # 带 YAML frontmatter 的流程说明书
```

13.5 Codex Security：按已安装能力操作

Codex Security 是独立的安全审查能力。请在客户端的插件目录中查看可用的 **Codex Security** 插件及其说明，再选择适合仓库的扫描入口；不同版本可能提供不同命令。本书不把未经核实的 Daybreak 模型路由、网络安全评级或扫描参数写成可执行事实。[Codex Security 文档]

```
codex plugin list
```

13.6 3D 实践示例：GPT-6 Astra、Tripo3D 与 Blender

以下是开发者可自行搭建的工作流示例：用 Astra 整理资产要求，用 Tripo3D 等 3D 服务生成 GLB，再用 Blender 的 `bpy` 脚本做渲染检查。各环节是独立工具，**不代表 OpenAI 与第三方有官方集成，也不保证生成质量或完成时间**。目前引用的 OpenAI 官方 GPT-6 资料没有给出本书旧版所写的 BenchCAD 数字。[GPT-6 官方指南]

1. 工作流边界

模型负责整理要求和辅助编写脚本；3D 服务负责生成网格；Blender 负责导入和渲染。最后由开发者审查几何、材质和性能，再决定是否用于产品。

```
创意和比例要求 → GPT-6 Astra 整理提示词 → Tripo3D 生成 GLB
                                          ↓
                        Blender bpy 渲染与人工检查 → Three.js 展示
```

2. 四步操作演练

步骤一：使用 GPT-6 Astra 结构化提纯 3D 提示词

在 Codex 终端或桌面端中向 GPT-6 Astra 发送需求，要求其以 3D 技术总监角色输出适合 Tripo3D 的提示词：

```
你现在是资深 3D 技术美术总监。我要在 Web 项目中嵌入一个“赛博朋克风格的全息贩卖机”。
请输出一份专为 Tripo3D 优化的专业结构化 3D 生成提示词。

要求：
1. 突出主体轮廓（Silhouette）与几何对称性；
2. 强制指定拓扑质量标签（Clean quad topology, game-ready, low-poly）；
3. 声明 PBR 材质映射规范（Matte painted metal, glowing neon emissive panels）。
```

例如，可整理出以下供 3D 服务参考的提示词；实际生成结果需检查：

```
Cyberpunk holographic vending machine, freestanding rectangular kiosk with chamfered edges.
Center features transparent acrylic dispenser bay with internal glowing LED strip.
Materials: Brushed matte dark gray titanium alloy (roughness: 0.35, metallic: 0.85),
glowing cyan neon emissive trim lines (emission strength: 5.0), scratched hazard decals on base.
Topology tags: clean quad-dominant topology, watertight mesh, manifold geometry, PBR 4K
textures, game-ready asset.
```

步骤二：Tripo3D 快速生成并导出标准化 GLB

在 [Tripo3D] 等服务中输入提示词，检查生成的网格和材质，再导出为 **`vending_machine.glb`**，放入项目的 `public/models/` 目录。生成速度和拓扑质量取决于服务与输入。

步骤三：GPT-6 Astra 编写 Blender 自动化管线脚本（`pipeline.py`）

无需人工打开 Blender 费时调整，让 Astra 编写并运行完全无头的 Python 自动化脚本：

```
import bpy
import math
```

```
from pathlib import Path

# 1. 初始化纯净环境（清空默认立方体与杂乱灯光）
bpy.ops.wm.read_factory_settings(use_empty=True)

# 2. 自动导入 Tripo3D 生成的 GLB 资产
asset_path = Path("public/models/vending_machine.glb").resolve()
if not asset_path.is_file():
    raise FileNotFoundError(asset_path)
bpy.ops.import_scene.gltf(filepath=str(asset_path))

# 3. 选中导入网格，自动计算边界盒并吸附至地面（Z=0）
imported_objs = [obj for obj in bpy.context.selected_objects if obj.type == 'MESH']
if imported_objs:
    bpy.ops.object.select_all(action='DESELECT')
    for obj in imported_objs:
        obj.select_set(True)
    bpy.context.view_layer.objects.active = imported_objs[0]
    bpy.ops.object.origin_set(type='ORIGIN_GEOMETRY', center='BOUNDS')

    # 获取世界坐标系下的最低点并补偿对齐
    min_z = min([v[2] for obj in imported_objs for v in [obj.matrix_world @ v.co for v in
obj.data.vertices]])
    for obj in imported_objs:
        obj.location.z -= min_z

# 4. 自动化标准影棚三点布光（Studio 3-Point Lighting）
# 主光（Key Light）
key_light_data = bpy.data.lights.new(name="Key_Light", type='AREA')
key_light_data.energy = 500
key_light_data.size = 2.0
key_light = bpy.data.objects.new(name="Key_Light", object_data=key_light_data)
key_light.location = (3.0, -3.0, 4.0)
bpy.context.collection.objects.link(key_light)

# 辅光（Fill Light）
fill_light_data = bpy.data.lights.new(name="Fill_Light", type='AREA')
fill_light_data.energy = 200
fill_light_data.size = 3.0
fill_light = bpy.data.objects.new(name="Fill_Light", object_data=fill_light_data)
fill_light.location = (-3.0, -2.0, 2.0)
bpy.context.collection.objects.link(fill_light)

# 轮廓光（Rim Light）
rim_light_data = bpy.data.lights.new(name="Rim_Light", type='SUN')
rim_light_data.energy = 3.0
rim_light = bpy.data.objects.new(name="Rim_Light", object_data=rim_light_data)
rim_light.location = (0.0, 4.0, 3.0)
rim_light.rotation_euler = (math.radians(-45), 0, math.radians(180))
bpy.context.collection.objects.link(rim_light)

# 5. 设置展示摄像机
cam_data = bpy.data.cameras.new("Product_Camera")
cam_obj = bpy.data.objects.new("Product_Camera", cam_data)
cam_obj.location = (0, -4.5, 2.0)
cam_obj.rotation_euler = (math.radians(72), 0, 0)
```

```
bpy.context.collection.objects.link(cam_obj)
bpy.context.scene.camera = cam_obj

# 6. 配置 Eevee 快速无头验证渲染
bpy.context.scene.render.engine = 'BLENDER_EEVEE_NEXT'
bpy.context.scene.render.resolution_x = 1280
bpy.context.scene.render.resolution_y = 720
render_path = Path("public/renders/validation_preview.png").resolve()
render_path.parent.mkdir(parents=True, exist_ok=True)
bpy.context.scene.render.filepath = str(render_path)

bpy.ops.render.render(write_still=True)
print("✅ Blender 自动化巡检渲染完成：./public/renders/validation_preview.png")
```

在终端以无头后台方式运行：

```
blender --background --python pipeline.py
```

渲染完成后，把 `validation_preview.png` 提供给具备图像输入的 Codex 会话，请它检查可见的模型和光照问题；法线、UV 与性能仍需在 Blender 中人工核实。

步骤四：一键嵌入 Web 端（React Three Fiber 商业化呈现）

最后，将该资产嵌入你的前端产品（如 Next.js 15 或 Vite 项目）中，供用户实时旋转查看：

```
// components/VendingMachineViewer.tsx
import { Canvas } from '@react-three/fiber';
import { useGLTF, OrbitControls, Environment } from '@react-three/drei';

export function VendingMachineViewer() {
  const { scene } = useGLTF('/models/vending_machine.glb');
  return (
    <div className="w-full h-[500px] bg-slate-900 rounded-xl overflow-hidden shadow-2xl">
      <Canvas camera={{ position: [0, 2, 4], fov: 45 }}>
        <ambientLight intensity={0.7} />
        <directionalLight position={[5, 10, 5]} intensity={1.2} />
        <primitive object={scene} position={[0, -1, 0]} />
        <Environment preset="city" />
        <OrbitControls autoRotate autoRotateSpeed={2.0} enableZoom={true} />
      </Canvas>
    </div>
  );
}
```

13.7 成本优化与弹性分流

日常复杂编码从 `gpt-6.1-sol` 开始；范围明确的提取或小改动选 `gpt-6-luna`；跨应用、需要更高判断能力的任务评估 `gpt-6-astra`。先在自己的账号和客户端核对可用模型，再根据实际质量、时延和成本调整。Codex 模型说明

若项目使用不同供应商，先独立验证其会话格式与工具能力。配套的 Codex Switch 属于第三方工具，其兼容性以该项目当前文档和实测为准。

13.8 本章迁移清单

```
codex --version
npm install -g @openai/codex@latest
codex --version
codex --model gpt-6.1-sol
codex exec --sandbox workspace-write "Review the current changes and report issues"
codex plugin list
```

1. 根据 Codex 模型说明核对账号和工作区的 GPT-6 可用性。
2. 搜索保存的 `gpt-5.4` 、`gpt-5.4-mini` 和 `--full-auto` 配置；按各自适用范围迁移。
3. 若使用 Hooks，对照 Hooks 文档检查事件结构，并用 `/hooks` 审查信任状态。

排错速查

现象	检查动作
模型选择器没有 GPT-6.1 Sol 或 Luna	核对 CLI 版本、账号套餐、推出进度和工作区管理员设置
`--full-auto` 显示弃用警告	新的非交互调用改用 `--sandbox workspace-write`；旧入口仍保留兼容
配置的模型无法使用	区分 Codex 登录态与 API 权限，在 `/model` 选择当前可用模型
Hooks 未执行	检查事件数组结构，并在 `/hooks` 查看是否需要信任

13.9 不随版本变化的原则

将可能变化的模型 ID 和 CLI 选项集中管理；每次升级先对照官方文档，再验证本机行为。目标、权限边界与验收条件仍应写清楚。

关联工具频道

查看 PMER Codex 工具频道的版本演进，快速了解近期变化与上手提醒；本章提供详细配置。两边资料均核对至 2026 年 10 月 21 日。
